11

Name Resolution and the Domain Name System (DNS)

11.1 Introduction

The protocols we have studied so far operate using IP addresses to identify the hosts that participate in a distributed application. These addresses (especially IPv6 addresses) are cumbersome for humans to use and remember, so the Internet supports the use of *host names* to identify hosts, both clients and servers. In order to be used by protocols such as TCP and IP, host names are converted into IP addresses using a process known as *name resolution*. There are different forms of name resolution in the Internet, but the most prevalent and important one uses a distributed database system known as the *Domain Name System* (DNS) [MD88]. DNS runs as an application on the Internet, using IPv4 or IPv6 (or both). For scalability, DNS names are hierarchical, as are the servers that support name resolution.

DNS is a distributed client/server networked database that is used by TCP/IP applications to map between host names and IP addresses (and vice versa), to provide electronic mail routing information, service naming, and other capabilities. We use the term distributed because no single site on the Internet knows all of the information. Each site (university department, campus, company, or department within a company, for example) maintains its own database of information and runs a server program that other systems across the Internet (clients) can query. The DNS provides the protocol that allows clients and servers to communicate with each other and also a protocol for allowing servers to exchange information.

From an application's point of view, access to the DNS is through an application library called a *resolver*. In general, an application must convert a host name to an IPv4 and/or IPv6 address before it can ask TCP to open a connection or send a unicast datagram using UDP. The TCP and IP protocol implementations know nothing about the DNS; they operate only with the addresses.

In this chapter we will take a look at how the names in DNS are set up, how resolvers and servers communicate using the Internet protocols (mainly UDP), and some of the other resolution mechanisms that are used in Internet environments. We do not cover all of the administrative details of running a name server or all of the options available with resolvers and servers. Such information is available from various other sources, including Albitz and Liu's *DNS and BIND* text [AL06] and in [RFC6168]. We discuss the details of DNS security (DNSSEC) in Chapter 18.

11.2 The DNS Name Space

The set of all names used with DNS constitutes the DNS *name space*. This space is partitioned hierarchically and is case insensitive, similar to computer file system folders (directories) and files. The current DNS name space is a tree of domains with an unnamed root at the top. The top echelons of the tree are the so-called top-level domains (TLDs), which include *generic TLDs* (gTLDs), *country-code TLDs* (ccTLDs), and *internationalized country-code TLDs* (IDN ccTLDs), plus a special *infrastructure TLD* called, for historical reasons, *ARPA* [RFC3172]. These form the top levels of a naming tree with the form shown in Figure 11-1.

There are five commonly used groups of TLDs, and one group of specialized domains being used for *internationalized domain names* (IDNs).[1] The history of IDNs, one piece of the "internationalization" or "i18n" of the Internet, is long and somewhat complicated. Across the world, there are multiple languages, and each uses one or more written scripts. While the Unicode standard [U11] aims to capture the entire set of characters, many characters look the same but have different Unicode values. Furthermore, characters written as text may flow from right to left, left to right, or (when combining certain texts with others) in both directions. Couple these (and other) somewhat technical concerns with concerns regarding equity and international law and politics, and a considerable hurdle results. The interested reader may wish to consult the IAB's review of IDNs [RFC4690], published in 2006, for more information. Current information is available from [IIDN].

The gTLDs are grouped into categories: *generic*, *generic-restricted*, and *sponsored*. The generic gTLDs (*generic* appears twice) are open for unrestricted use. The others (generic-restricted and sponsored) are limited to various sorts of uses or are constrained as to what entity may assign names from the domain. For example, EDU is used for educational institutions, MIL and GOV are used for military and government institutions of the United States, and INT is used for international organizations (such as NATO). Table 11-1 provides a summary of the 22 gTLDs from [GTLD] as of mid-2011. There is a "new gTLD" program in the works that may significantly expand the current set, possibly to several hundred or even thousand. This program and policies relating to TLD management in general are maintained by the Internet Corporation for Assigned Names and Numbers (ICANN) [ICANN].

1. Figure 11-1 also shows 11 test IDN domains, which are still available.

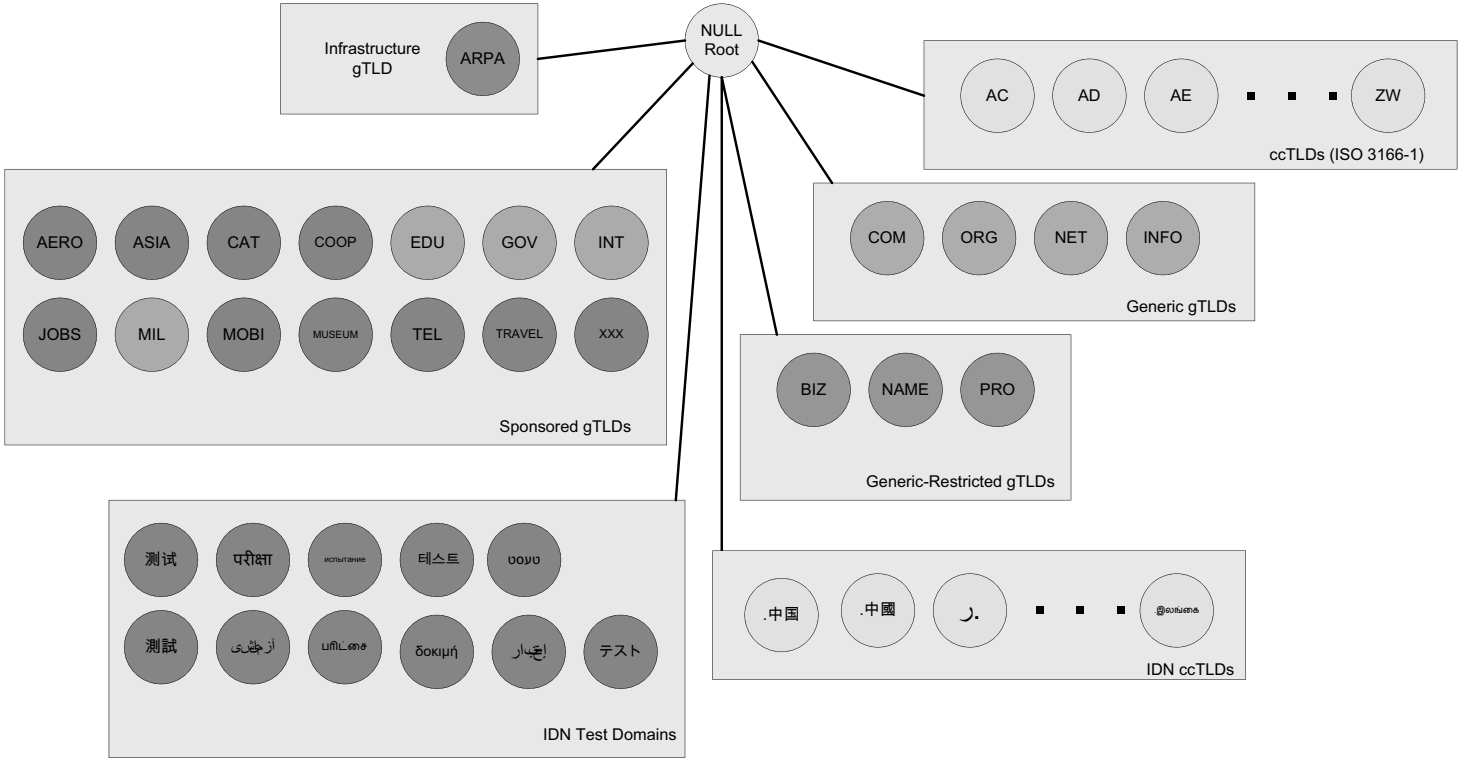

Figure 11-1 The DNS name space forms a hierarchy with an unnamed root at the top. The top-level domains (TLDs) include generic TLDs (gTLDs), country-code TLDs (ccTLDs), internationalized TLDs (IDN ccTLDs), and a special infrastructure TLD called *ARPA*.

Table 11-1 The generic top-level domains (gTLDs), circa 2011

TLD	First Use (est.)	Use	Example
AERO	December 21, 2001	Air-transport industry	www.sita.aero
ARPA	January 1, 1985	Infrastructure	18.in-addr.arpa
ASIA	May 2, 2007	Pan-Asia and Asia Pacific	www.seo.asia
BIZ	June 26, 2001	Business uses	neustar.biz
CAT	December 19, 2005	Catalan linguistic/cultural community	www.domini.cat
COM	January 1, 1985	Generic	icanhascheezburger.com
COOP	December 15, 2001	Cooperative associations	www.ems.coop
EDU	January 1, 1985	Post-secondary educational institutions recognized by U.S.A.	hpu.edu
GOV	January 1, 1985	U.S. government	whitehouse.gov
INFO	June 25, 2001	Generic	germany.info
INT	November 3, 1988	International treaty organizations	nato.int
JOBS	September 8, 2005	Human resource managers	intel.jobs
MIL	January 1, 1985	U.S. military	dtic.mil
MOBI	October 30, 2005	Customers/providers of mobile products/services	flowers.mobi
MUSEUM	October 30, 2001	Museums	icom.museum
NAME	August 16, 2001	Individuals	www.name
NET	January 1, 1985	Generic	ja.net
ORG	December 9, 2002	Generic	slashdot.org
PRO	May 6, 2002	Credentialed professionals/entities	nic.pro
TEL	March 1, 2007	Contact data for businesses/ individuals	telnic.tel
TRAVEL	July 27, 2005	Travel industry	cancun.travel
XXX	April 15, 2011	Adult entertainment industry	whois.nic.xxx

The ccTLDs include the two-letter country codes specified by the ISO 3166 standard [ISO3166], plus five that are not: `uk`, `su`, `ac`, `eu`, and `tp` (the last one is being phased out). Because some of these two-letter codes are suggestive of other uses and meanings, various countries have been able to find commercial windfalls from selling names within their ccTLDs. For example, the domain name `cnn.tv` is really a registration in the Pacific island of Tuvalu, which has been selling domain names associated with the television entertainment industry. Creating a name in such an unconventional way is sometimes called a *domain hack*.

11.2.1 DNS Naming Syntax

The names below a TLD in the DNS name tree are further partitioned into groups known as *subdomains*. This is very common practice, especially for the ccTLDs. For

example, most educational sites in England use the suffix `.ac.uk`, whereas names for most for-profit companies there end in the suffix `.co.uk`. In the United States, city government Web sites tend to use the subdomain `ci.`*`city`*`.`*`state`*`.us` where *state* is the two-letter abbreviation for the name of the state and *city* is the name of the city. For example, the site `www.ci.manhattan-beach.ca.us` is the site of Manhattan Beach, California's, city government in the United States.

The example names we have seen so far are known as fully qualified domain names (FQDNs). They are sometimes written more formally with a trailing period (e.g., `mit.edu.`). This trailing period indicates that the name is complete; no additional information should be added to the name when performing a name resolution. In contrast to the FQDN, an *unqualified domain name*, which is used in combination with a default domain or domain search list set during system configuration, has one or more strings appended to the end. When a system is configured (see Chapter 6), it is typically assigned a default domain extension and search list using DHCP (or, less commonly, the RDNSS and DNSSL RA options). For example, the default domain `cs.berkeley.edu` might be configured in systems at the computer science department at UC Berkeley. If a user on one of these machines types in the name `vangogh`, the local resolver software converts this name to the FQDN `vangogh.cs.berkeley.edu.` before invoking a resolver to determine `vangogh`'s IP address.

A domain name consists of a sequence of *labels* separated by periods. The name represents a location in the name hierarchy, where the period is the hierarchy delimiter and descending down the tree takes place from right to left in the name. For example, the FQDN

```
www.net.in.tum.de.
```

contains a host name label (`www`) in a four-level-deep domain (`net.in.tum.de`). Starting from the root, and working from right to left in the name, the TLD is `de` (the ccTLD for Germany), `tum` is shorthand for Technische Universität München, `in` is shorthand for *informatik* (German for "computer science"), and finally `net` is shorthand for the networks group within the computer science department. Labels are case-insensitive for matching purposes, so the name `ACME.COM` is equivalent to `acme.com` or `AcMe.cOm` [RFC4343]. Each label can be up to 63 characters long, and an entire FQDN is limited to at most 255 (1-byte) characters. For example, this domain name:

```
thelongestdomainnameintheworldandthensomeandthensomemoreandmore.com
```

was allegedly submitted as a potential world record for the longest name, with a label of length 63, but was judged to have been of insufficient merit to justify a place in the Guinness World Records.

The hierarchical structure of the DNS name space allows different administrative authorities to manage different parts of the name space. For example, creating

a new DNS name of the form `elevator.cs.berkeley.edu` would likely require dealing with the owner of the `cs.berkeley.edu` subdomain only. The `berkeley .edu` and `edu` portions of the name space would not require alteration, so the owners of those would not need to be bothered. This feature of DNS is one key aspect of its *scalability*. That is, no single entity is required to administer all the changes for the entire DNS name space. Indeed, creating a hierarchical structure for names was one of the first responses in the Internet community to the pressures of scaling and a major motivator for the structure used today. The original Internet naming scheme was flat (i.e., no hierarchy), and a single entity was responsible for assigning, maintaining, and distributing the list of nonconflicting names. Over time, as more names were required and more changes were being made, this approach became unworkable [MD88].

11.3 Name Servers and Zones

Management responsibility for portions of the DNS name space is assigned to individuals or organizations. A person given responsibility for managing part of the active DNS name space (one or more domains) is supposed to arrange for at least two *name servers* or *DNS servers* to hold information about the name space so that users of the Internet can perform queries on the names. The collection of servers forms the DNS (service) itself, a distributed system whose primary job is to provide name-to-address mappings. However, it can also provide a wide array of additional information.

The unit of administrative delegation, in the language of DNS servers, is called a *zone*. A zone is a subtree of the DNS name space that can be administered separately from other zones. Every domain name exists within some zone, even the TLDs that exist in the *root zone*. Whenever a new record is added to a zone, the DNS administrator for the zone allocates a name and additional information (usually an IP address) for the new entry and enters these into the name server's database. At a small campus, for example, one person could do this each time a new server is added to the network, but in a large enterprise the responsibility would have to be delegated (probably by departments or other organizational units), as one person likely could not keep up with the work.

A DNS server can contain information for more than one zone. At any hierarchical change point in a domain name (i.e., wherever a period appears), a different zone and containing server may be accessed to provide information for the name. This is called a *delegation*. A common delegation approach uses a zone for implementing a second-level domain name, such as `berkeley.edu`. In this domain, there may be individual hosts (e.g., `www.berkeley.edu`) or other domains (e.g., `cs.berkeley.edu`). Each zone has a designated owner or responsible party who is given authority to manage the names, addresses, and subordinate zones within the zone. Often this person manages not only the contents of the zone but also the name servers that contain the zone's database(s).

Zone information is supposed to exist in at least two places, implying that there should be at least two servers containing information for each zone. This is for redundancy; if one server is not functioning properly, at least one other server is available. All of these servers contain identical information about a zone. Typically, among the servers, a *primary* server contains the zone database in a disk file, and one or more *secondary* servers obtain copies of the database in its entirety from the primary using a process called a *zone transfer*. DNS has a special protocol for performing zone transfers, but copies of a zone's contents can also be obtained using other means (e.g., the `rsync` utility [RSYNC]).

11.4 Caching

Name servers contain information such as name-to-IP-address mappings that may be obtained from three sources. The name server obtains the information directly from the zone database, as the result of a zone transfer (e.g., for a slave server), or from another server in the course of processing a resolution. In the first case, the server is said to contain *authoritative* information about the zone and may be called an *authoritative server* for the zone. Such servers are identified by name within the zone information.

Most name servers (except some of the root and TLD servers) also *cache* zone information they learn, up to a time limit called the *time to live* (TTL). They use this cached information to answer queries. Doing so can greatly decrease the amount of DNS message traffic that would otherwise be carried on the Internet [J02]. When answering a query, a server indicates whether the information it is returning has been derived from its cache or from its authoritative copy of the zone. When cached information is returned, it is common for a server to also include the domain names of the name servers that can be contacted to retrieve authoritative information about the corresponding zone.

As we shall see, each DNS record (e.g., name-to-IP-address mapping) has its own TTL that controls how long it can be cached. These values are set and altered by the zone administrator when necessary. The TTL dictates how long a mapping can be cached anywhere within DNS, so if a zone changes, there still may exist cached data within the network, potentially leading to incorrect DNS resolution behavior until expiry of the TTL. For this reason, some zone administrators, anticipating a change to the zone contents, first reduce the TTL before implementing the change. Doing so reduces the window for incorrect cached data to be present in the network.

It is worth mentioning that caching is applied both for successful resolutions and for unsuccessful resolutions (called *negative caching*). If a request for a particular domain name fails to return a record, this fact is also cached. Doing so can help to reduce Internet traffic when errant applications repeatedly make requests for names that do not exist. Negative caching was changed from optional to mandatory by [RFC2308].

In some network configurations (e.g., those using older UNIX-compatible systems), the cache is maintained in a nearby name server, not in the resolvers resident in the clients. Placing the cache in the server allows any hosts on the LAN that use the nearby server to benefit from the server's cache but implies a small delay in accessing the cache over the local network. In Windows and more recent systems (e.g., Linux), the client can maintain a cache, and it is made available to all applications running on the same system. In Windows, this happens by default, and in Linux, it is a service that can be enabled or disabled.

On Windows, the local system's cache parameters may be modified by editing the following registry entry:

```
HKLM\SYSTEM\CurrentControlSet\Services\DNSCache\Parameters
```

The DWORD value `MaxNegativeCacheTtl` gives the maximum number of seconds that a negative DNS result remains in the resolver cache. The DWORD value `MaxCacheTtl` gives the maximum number of seconds that a DNS record may remain in the resolver cache. If this value is less than the TTL of a received DNS record, the lesser value controls how long the record remains in cache. These two registry keys do not exist by default, so they must be created in order to be used.

In Linux and other systems that support it, the *Name Service Caching Daemon* (NSCD) provides a client-side caching capability. It is controlled by the `/etc/nscd.conf` file that can indicate which types of resolutions (for DNS and some other services) are cached, along with some cache parameters such as TTL settings. In addition, the file `/etc/nsswitch.conf` controls how name resolution for applications takes place. Among other things, it can control whether local files, the DNS protocol (see Section 11.5), and/or NSCD is employed for mappings.

11.5 The DNS Protocol

The DNS protocol consists of two main parts: a query/response protocol used for performing queries against the DNS for particular names, and another protocol for name servers to exchange database records (zone transfers). It also has a way to notify secondary servers that the zone database has evolved and a zone transfer is necessary (DNS Notify), and a way to dynamically update the zone (dynamic updates). By far, the most typical usage is a simple query/response to look up the IPv4 address that corresponds to a domain name.

Most often, DNS name resolution is the process of mapping a domain name to an IPv4 address, although IPv6 addresses mappings work in essentially the same way. DNS query/response operations are supported over the distributed DNS infrastructure consisting of servers deployed locally at each site or ISP, and a special set of *root servers*. There is also a special set of *generic top-level domain servers*

used for scaling some of the larger gTLDs, including COM and NET. As of mid-2011, there are 13 root servers named by the letters *A* through *M* (see [ROOTS] for more information about them); 9 of them have IPv6 addresses. There are also 13 gTLD servers, also labeled *A* through *M*; 2 of them have IPv6 addresses. By contacting a root server and possibly a gTLD server, the name server for any TLD in the Internet can be discovered. These servers are mutually coordinated to provide the same information. Some of them are not a single physical server but instead a group of servers (over 50 for the *J* root server) that use the same IP address (i.e., using IP anycast addressing; see Chapter 2).

A full resolution that is unable to benefit from preexisting cached entries takes place among several entities, as shown in Figure 11-2.

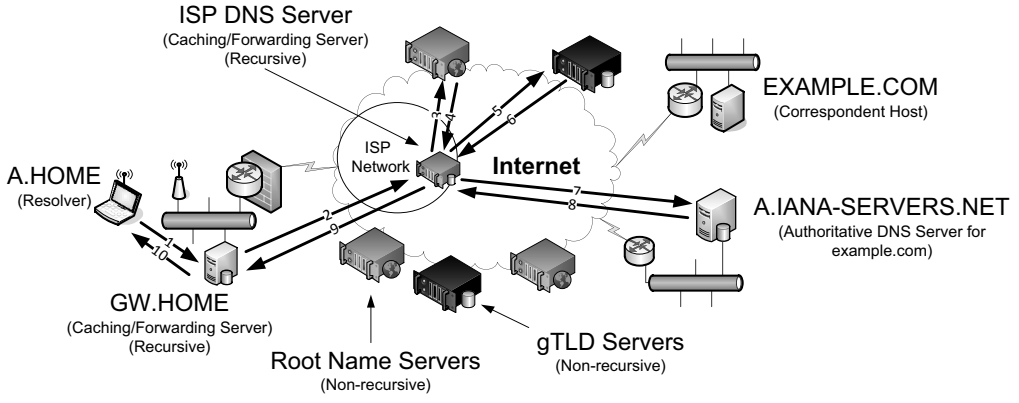

Figure 11-2 A typical recursive DNS query for `EXAMPLE.COM` from `A.HOME` involves up to ten messages. The local recursive server (`GW.HOME` here) uses a DNS server provided by its ISP. That server, in turn, uses an Internet root name server and a gTLD server (for `COM` and `NET` TLDs) to find the name server for the `EXAMPLE.COM` domain. That name server (`A.IANA-SERVERS.NET` here) provides the required IP address for the host `EXAMPLE.COM`. All of the recursive servers cache any information learned for later use.

Here, we have a laptop called `A.HOME` residing nearby the DNS server `GW.HOME`. The domain `HOME` is private, so it is not known to the Internet—only locally at the user's residence. When a user on `A.HOME` wishes to connect to the host `EXAMPLE.COM` (e.g., because a Web browser has been instructed to access the page `http://EXAMPLE.COM`), `A.HOME` must determine the IP address for the server `EXAMPLE.COM`. Assuming it does not know this address already (it might if it has accessed the host recently), the resolver software on `A.HOME` first makes a request to its local name server, `GW.HOME`. This is a request to convert the name `EXAMPLE.COM` into an address and constitutes message 1 (labeled on an arrow in Figure 11-2).

Note

If the A.HOME system is configured with a default domain search list, there may be additional queries. For example, if .HOME is a default search domain used by A.HOME, the first DNS query may be for the name EXAMPLE.COM.HOME, which will fail at the GW.HOME name server, which is authoritative for .HOME. A subsequent query will typically remove the default extension, resulting in a query for EXAMPLE.COM.

If GW.HOME does not already know the IP address for EXAMPLE.COM or the name servers for either the EXAMPLE.COM domain or the COM TLD, it forwards the request to another DNS server (called *recursion*). In this case, a request (message 2) goes to an ISP-provided DNS server. Assuming that this server also does not know the required address or other information, it contacts one of the root name servers (message 3). The root servers are not recursive, so they do not process the request further but instead return the information required to contact a name server for the COM TLD. For example, it might return the name A.GTLD-SERVERS .NET and one or more of its IP addresses (message 4). With this information, the ISP-provided server contacts the gTLD server (message 5) and discovers the name and IP addresses of the name servers for the domain EXAMPLE.COM (message 6). In this case, one of the servers is A.IANA-SERVERS.NET.

Given the correct server for the domain, the ISP-provided server contacts the appropriate server (message 7), which responds with the requested IP address (message 8). At this point, the ISP-provided server can respond to GW.HOME with the required information (message 9). GW.HOME is now able to complete the initial query and responds to the client with the desired IPv4 and/or IPv6 address(es) (message 10).

From the perspective of A.HOME, the local name server was able to perform the request. However, what really happened is a *recursive query*, where the GW.HOME and ISP-provided servers in turn made additional DNS requests to satisfy A.HOME's query. In general, most name servers perform recursive queries such as this. The notable exceptions are the root servers and other TLD servers that do not perform recursive queries. These servers are a relatively precious resource, so encumbering them with recursive queries for every machine that performs a DNS query would lead to poor global Internet performance.

11.5.1 DNS Message Format

There is one basic DNS message format [RFC6195]. It is used for all DNS operations (queries, responses, zone transfers, notifications, and dynamic updates), as illustrated in Figure 11-3.

The basic DNS message begins with a fixed 12-byte header followed by four variable-length *sections*: questions (or queries), answers, authority records, and additional records. All but the first section contain one or more *resource records*

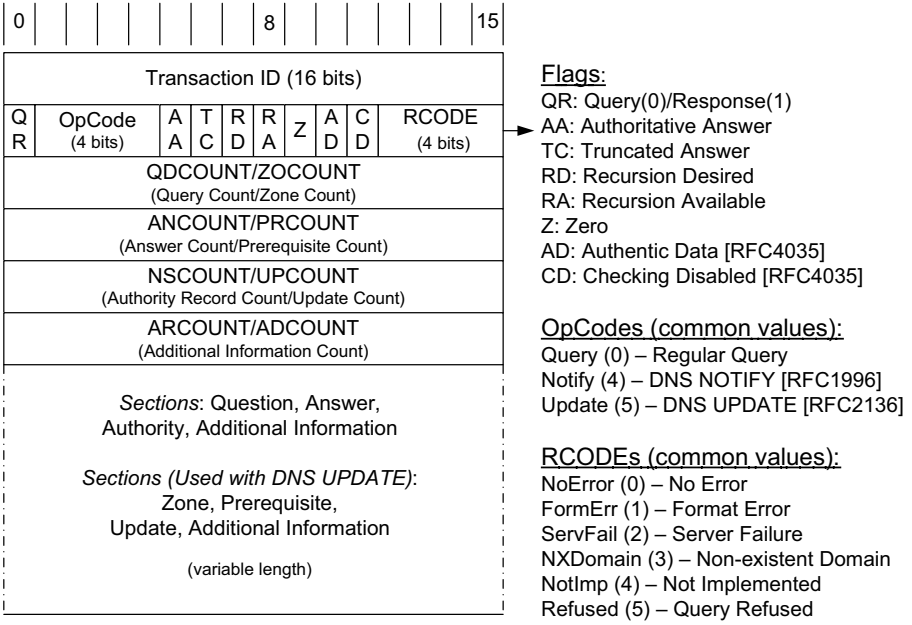

Figure 11-3 The DNS message format has a fixed 12-byte header. The entire message is usually carried in a UDP/IPv4 datagram and limited to 512 bytes. DNS UPDATE (DNS with dynamic updates) uses the field names *ZOCOUNT*, *PRCOUNT*, *UPCOUNT*, and *ADCOUNT*. A special extension format (called EDNS0) allows messages to be larger than 512 bytes, which is required for DNSSEC (see Chapter 18).

(RRs), which we discuss in detail in Section 11.5.6. (The question section contains a data item that is very close in structure to an RR.) RRs can be cached; questions are not.

In the fixed-length header, the *Transaction ID* field is set by the client and returned by the server. It lets the client match responses to requests. The second 16-bit word includes a number of flags and other subfields. Beginning from the left-most bit, *QR* is a 1-bit field: 0 means the message is a query; 1 means it is a response. The next is the *OpCode*, a 4-bit field. The normal value is 0 (a standard query) for requests and responses. Other values are: 4 (notify), and 5 (update). Other values (1–3) are deprecated or never seen in operational use. Next is the *AA* bit field that indicates an "authoritative answer" (as opposed to a cached answer). *TC* is a 1-bit field that means "truncated." With UDP, this flag being set means that the total size of the reply exceeded 512 bytes, and only the first 512 bytes of the reply were returned.

RD is a bit field that means "recursion desired." It can be set in a query and is then returned in the response. It tells the server to perform a recursive query. If the bit is not set, and the requested name server does not have an authoritative answer, the requested name server returns a list of other name servers to contact

for the answer. At this point, the overall query may be continued by contacting the list of other name servers. This is called an *iterative query*. *RA* is a bit field that means "recursion available." This bit is set in the response if the server supports recursion. Root servers generally do *not* support recursion, thereby forcing clients to perform iterative queries to complete name resolution. The *Z* bit field must be 0 for now but is reserved for future use.

The *AD* bit field is set to true if the contained information is *authenticated*, and the *CD* bit is set to true if security checking is disabled (see Chapter 18). The *Response Code* (or *RCODE*) field is a 4-bit field with the return code whose possible values are given in [DNSPARAM]. The common values include 0 (no error) and 3 (name error or "nonexistent domain," written as NXDOMAIN). A list of the first 11 error codes is given in Table 11-2 (values 11 through 15 are unassigned). Additional types are defined using a special extension (see Section 11.5.2). A name error is returned only from an authoritative name server and means that the domain name specified in the query does not exist.

Table 11-2 The first ten error types used with the *RCODE* field

Value	Name	Reference	Description and Purpose
0	NoError	[RFC1035]	No error
1	FormErr	[RFC1035]	Format error; query cannot be interpreted
2	ServFail	[RFC1035]	Server failure; error in processing at server
3	NXDomain	[RFC1035]	Nonexistent domain; unknown domain referenced
4	NotImp	[RFC1035]	Not implemented; request not supported in server
5	Refused	[RFC1035]	Refused; server unwilling to provide answer
6	YXDomain	[RFC2136]	Name exists but should not (used with updates)
7	YXRRSet	[RFC2136]	RRSet exists but should not (used with updates)
8	NXRRSet	[RFC2136]	RRSet does not exist but should (used with updates)
9	NotAuth	[RFC2136]	Server not authorized for zone (used with updates)
10	NotZone	[RFC2136]	Name not contained in zone (used with updates)

The next four fields are 16 bits in size and specify the number of entries in the question, answer, authority, and additional information sections that complete the DNS message. For a query, the number of questions is normally 1 and the other three counts are 0. For a reply, the number of answers is at least 1. Questions have a name, type, and class. (Class supports non-Internet records, but we ignore this for our purposes. The type identifies the type of object being looked up.) All of the other sections contain zero or more RRs. RRs contain a name, type, and class information, but also the TTL value that controls how long the data can be cached. We shall discuss the most important RR types in detail once we have a look at how DNS encodes names and selects which transport protocol to use when carrying DNS messages.

11.5.1.1 Names and Labels

The variable-length sections at the end of a DNS message contain a collection of questions, answers, authority information (names of name servers that contain authoritative information for certain data), and additional information that may be useful to reduce the number of necessary queries. Each question and each RR begins with a *name* (called the domain name or owning name) to which it refers. Each name consists of a sequence of *labels*. There are two categories of label types: *data labels* and *compression labels*. Data labels contain characters that constitute a label; compression labels act as pointers to other labels. Compression labels help to save space in a DNS message when multiple copies of the same string of characters are present across multiple labels.

11.5.1.2 Data Labels

Each data label begins with a 1-byte count that specifies the number of bytes that immediately follow. The name is terminated with a byte containing the value 0, which is a label with a length of 0 (the label of the root). For example, the encoding of the name `www.pearson.com` would be as shown in Figure 11-4.

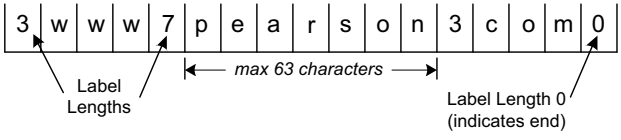

Figure 11-4 DNS names are encoded as a sequence of labels. This example encodes the name `www.pearson.com`, which (technically) has four labels. The end of the name is identified by a 0-length label of the nameless root.

For data labels, each label *Length* byte must be in the range of 0 to 63, as labels are limited to 63 bytes. No padding is used for labels, so the total name length could be odd. Although these labels are sometimes called "text" labels, they are capable of containing non-ASCII values. This use, however, is uncommon and not recommended. Indeed, even the internationalized domain names, which can encode Unicode characters [RFC5890][RFC5891], use a curious encoding syntax called "punycode" [RFC3492] that expresses Unicode characters using the ASCII character set. To be completely safe, it is recommended to follow the requirements in [RFC1035], which suggest that labels "start with a letter, end with a letter or digit, and have as interior characters only letters, digits and hyphen."

11.5.1.3 Compression Labels

In many cases, a DNS response carries information in the answer, authority, and additional information sections relating to the same domain name. If data labels were used, the same characters would be repeated in the DNS message when referring to the same name. To avoid this redundancy and save space, a compression

scheme is used. Anywhere the label portion of a domain name can occur, the single preceding count byte (which is normally between 0 and 63) instead has its 2 high-order bits turned on, and the remaining bits are combined with the bits in the subsequent byte to form a 14-bit pointer (offset) in the DNS message. The offset gives the number of bytes from the beginning of the DNS message where a data label (called the *compression target*) is to be found that should be substituted for the compression label. Compression labels are thus able to point to a location up to 16,383 bytes from the beginning. Figure 11-5 illustrates how we might encode the domain names `usc.edu` and `ucla.edu` using compression labels.

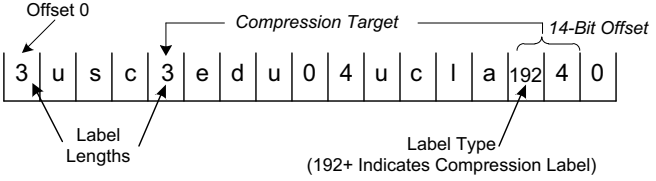

Figure 11-5 A compression label can reference other labels to save space. This is accomplished by setting the 2 high-order bits of the byte preceding the label contents. This signals that the following 14 bits are used in providing an offset for the replacement label. In this example, `usc.edu` and `ucla.edu` share the `edu` label.

In Figure 11-5 we see how the common label `edu` can be shared by the two domain names. Assuming the names start at offset 0, data labels are used to encode `usc.edu` as described previously. The next name is `ucla.edu`, and the label `ucla` is encoded using a data label. However, the label `edu` may be reused from the encoding of `usc.edu`. This is accomplished by setting the 2 high-order bits of the label *Type* byte to 1 and encoding the offset of `edu` in the remaining 14 bits. Because the first occurrence of `edu` is at offset 4, we only need to set the first byte to 192 (6 bits of 0) and the next byte to 4. The example in Figure 11-5 shows a savings of only 4 bytes, but it is clear how compression of larger common labels can result in more substantial savings.

11.5.2 The DNS Extension Format (EDNS0)

The basic DNS message format described so far can be restrictive in a number of ways. It has fixed-length fields, a total length limitation of 512 bytes when used with UDP (not including UDP or IP headers), and limited space (the 4-bit *RCODE* field) for indicating error types. An extension mechanism called *EDNS0* (because there could be future extensions beyond the index 0) is specified in [RFC2671]. While its use is not ubiquitous at present, it is necessary for supporting DNS security (DNSSEC; see Chapter 18), so it is likely to receive more widespread deployment over time.

EDNS0 specifies a particular type of RR (called an *OPT pseudo-RR* or *meta-RR*) that is added to the additional data section of a request or response to indicate the use of EDNS0. At most one such record may be present in any DNS message. We will discuss the particular format of an OPT pseudo-RR when we discuss the other RR types in Section 11.5.6. For now, the important thing to note is that if a UDP DNS message includes an OPT RR, it is permitted to exceed the 512-byte length limitation and may contain an expanded set of error codes.

EDNS0 also defines an extended label type (extending beyond the data labels and compression labels mentioned earlier). Extended labels have their first 2 bits in the label *Type/Length* byte set to 01, corresponding to values between 64 and 127 (inclusive). An experimental binary labeling scheme (type 65) was used at one time but is now not recommended. The value 127 is reserved for future use, and values above 127 are unallocated.

11.5.3 UDP or TCP

The well-known port number for DNS is 53, for both UDP and TCP. The most common format uses the UDP/IPv4 datagram structure shown in Figure 11-6.

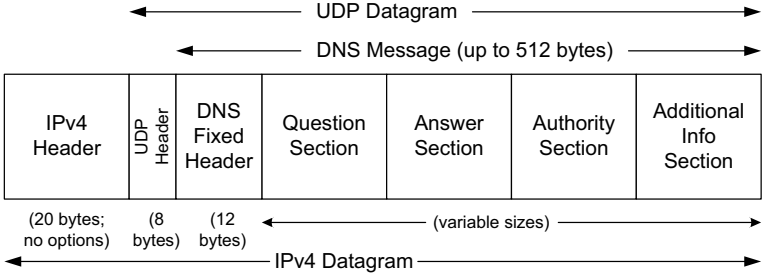

Figure 11-6 DNS messages are typically encapsulated in a UDP/IPv4 datagram and are limited to 512 bytes in size unless TCP and/or EDNS0 is used. Each section (except the question section) contains a set of resource records.

When a resolver issues a query and the response comes back with the *TC* bit field set ("truncated"), the size of the true response exceeded 512 bytes, so only the first 512 bytes are returned by the server. The resolver may issue the request again, using TCP, which now must be a supported configuration [RFC5966]. This allows more than 512 bytes to be returned because TCP breaks up large messages into multiple segments.

When a secondary name server for a zone starts up, it normally performs a zone transfer from the primary name server for the zone. Zone transfers can also be initiated by a timer or as a result of a DNS NOTIFY message (see Section 11.5.8.3). Full zone transfers use TCP as they can be large. Incremental zone transfers, where only the updated entries are transferred, may use UDP at first but switch to TCP if the response is too large, just like a conventional query.

When UDP is used, both the resolver and the server application software must perform their own timeout and retransmission. A recommendation for how to do this is given in [RFC1536]. It suggests starting with a timeout of at least 4s, and that subsequent timeouts result in an exponential increase of the timeout (a bit like TCP's algorithms; see Chapter 14). Linux and UNIX-like systems allow a change to be made to the retransmission timeout parameters by altering the contents of the `/etc/resolv.conf` file (by setting the `timeout` and `attempts` options).

11.5.4 Question (Query) and Zone Section Format

The question or query section of a DNS message lists the question(s) being referenced. The format of each question in the question section is shown in Figure 11-7. There is normally just one, although the protocol can support more. The same structure is also used for the zone section in dynamic updates (see Section 11.5.7), but with different names.

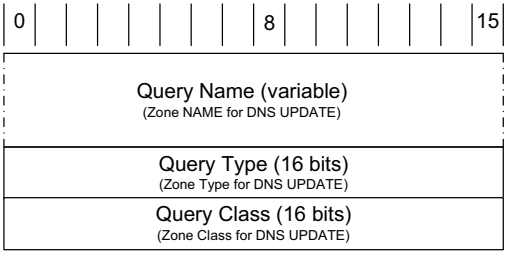

Figure 11-7 The query (or question) section of a DNS message does not contain a TTL because it is not cached.

The *Query Name* is the domain name being looked up, using the encoding for labels we described before. Each question has a *Query Type* and *Query Class*. The class value is 1, 254, or 255, indicating the Internet class, no class, or all classes, respectively, for all cases in which we are interested (other values are not typically used for TCP/IP networks). The *Query Type* field holds a value indicating the type of query being performed using the values from Table 11-2. The most common query type is A (or AAAA if IPv6 DNS resolution is enabled), which means that an IP address is desired for the query name. It is also possible to create a query of type ANY, which returns all RRs of any type in the same class that match the query name.

11.5.5 Answer, Authority, and Additional Information Section Formats

The final three sections in the DNS message, the answer, authority, and additional information sections, contain sets of RRs. RRs in these sections can, for the most part, have *wildcard* domain names as owning names. These are domain names in which the asterisk label—a data label containing only the asterisk character

[RFC4592]—appears first (i.e., leftmost). Each resource record has the form shown in Figure 11-8.

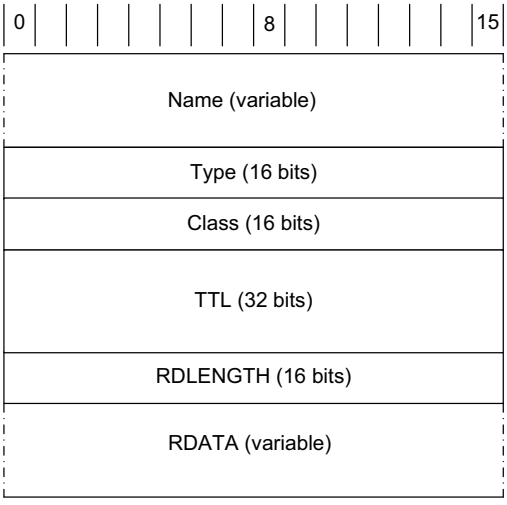

Figure 11-8 The format of a DNS resource record. For DNS in the Internet, the *Class* field always contains the value 1. The *TTL* field gives the maximum amount of time the RR can be cached (in seconds).

The *Name* field (sometimes called the "owning name," "owner," or "record owner's name") is the domain name to which the following resource data corresponds. It is in the same format we described earlier for names and labels. The *Type* field specifies one of the RR type codes (see Section 11.5.6). These are the same as the query type values we described earlier. The *Class* field is 1 for Internet data. The *TTL* field is the number of seconds for which the RR can be cached. The *Resource Data Length* (*RDLENGTH*) field specifies the number of bytes contained in the *Resource Data* (*RDATA*) field. The format of this data depends on the type. For example, A records (type 1) have a 32-bit IPv4 address in the RDATA area. We discuss other RR types later.

[RFC2181] defines the term *Resource Record Set* (RRSet) to be a set of resource records that share the same name, class, and type but not the same data. This occurs, for example, when a host has more than one address record for its name (e.g., because it has more than one IP address). TTLs for RRs in the same RRSet must be equal.

11.5.6 Resource Record Types

Although DNS is most commonly used to determine the IP address(es) that correspond to a particular name, it can also be used for the opposite purpose and for a number of other things. It can be used with both IPv4 and IPv6 and can even provide a distributed database function for other than Internet data (other classes,

in DNS terminology [RFC6195]). The wide range of capabilities provided by DNS is largely attributable to its ability to have different types of resource records.

There are many types of resource records (see [DNSPARAMS] for the complete list), and a single name may have multiple matching RRs. Table 11-3 provides a listing of the most common RR types used with conventional DNS (i.e., DNS without the DNSSEC security extensions).

Table 11-3 The popular resource record and query types used in DNS protocol messages. Additional records (not shown) are used when DNS security (DNSSEC) is employed.

Value	RR Type	Reference	Description and Purpose
1	A	[RFC1035]	Address record for IPv4 (32-bit IPv4 address)
2	NS	[RFC1035]	Name server; provides name of authoritative name server for zone
5	CNAME	[RFC1035]	Canonical name; maps one name to another (to provide a form of name aliasing)
6	SOA	[RFC1035]	Start of authority; provides authoritative information for the zone (name servers, e-mail address of contact, serial number, zone transfer timers)
12	PTR	[RFC1035]	Pointer; provides address to (canonical) name mapping; used with `in-addr.arpa` and `ip6.arpa` domains for IPv4 and IPv6 reverse queries
15	MX	[RFC1035]	Mail exchanger; provides name of e-mail handling host for a domain
16	TXT	[RFC1035] [RFC1464]	Text; provides a variety of information (e.g., used with SPF anti-spam scheme to identify authorized e-mail servers)
28	AAAA	[RFC3596]	Address record for IPv6 (128-bit IPv6 address)
33	SRV	[RFC2782]	Server selection; transport endpoints of a generic service
35	NAPTR	[RFC3403]	Name authority pointer; supports alternative name spaces
41	OPT	[RFC2671]	Pseudo-RR; supports larger datagrams, labels, return codes in EDNS0
251	IXFR	[RFC1995]	Incremental zone transfer
252	AXFR	[RFC1035] [RFC5936]	Full zone transfer; carried over TCP
255	(ANY)	[RFC1035]	Request for all (any) records

Resource records are used for many purposes but can be divided into three broad categories: data types, query types, and meta types. Data types are used to convey information stored in the DNS such as IP addresses and the names of authoritative name servers. Query types use the same values as data types, with a few additional values (e.g., AXFR, IXFR, and *). They can be used in the question section we described previously. Meta types designate transient data associated with a particular single DNS message. The OPT RR is the only meta type we

discuss in this chapter (all others are covered in Chapter 18). The most common data-type RRs include A, NS, SOA, MX, CNAME, PTR, TXT, AAAA, SRV, and NAPTR. The NS records are used to relate the DNS name space to the servers that perform resolution, and they contain the names of authoritative name servers for a zone. The A and AAAA records are used to provide an IPv4 or IPv6 address, respectively, given a particular name. The CNAME record provides a way to have an alias for another domain name. SRV and NAPTR records help applications to discover the location of servers supporting particular services, and to use alternative naming schemes (beyond DNS) to access such services. We shall explore each of these record types in the following sections.

11.5.6.1 Address (A, AAAA) and Name Server (NS) Records

Arguably, the most important records within DNS are the address (A, AAAA) and name server (NS) records. The A records contain 32-bit IPv4 addresses, and AAAA (called "quad-A") records contain IPv6 addresses. An NS record contains the name of an authoritative DNS server that contains information for a particular zone. Because the name of a DNS server alone is not sufficient to perform a query, the IP address(es) of these servers is also typically provided as a so-called glue record in the additional information section of DNS responses. Indeed, such glue records are required to avoid loops whenever the names of the authoritative name servers use the same domain name for which they are authoritative. (Consider how `ns1.example.com` would be resolved if the name server for `example.com` was `ns1.example.com`.) We can see the structure of A, AAAA, and NS records using the `dig` tool provided on most Linux/UNIX-like systems. Here, we make a request for records of any type associated with the domain name `rfc-editor.org`:

```
Linux% dig +nostats -t ANY rfc-editor.org

; <<>> DiG 9.6.0-P1 <<>> +nostats -t ANY rfc-editor.org
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 53052
;; flags: qr rd ra; QUERY: 1, ANSWER: 12, AUTHORITY: 0, ADDITIONAL: 2

;; QUESTION SECTION:
;rfc-editor.org.   IN ANY

;; ANSWER SECTION:
...
rfc-editor.org.  1654 IN AAAA 2001:1890:1112:1::2f
rfc-editor.org.  1654 IN A 64.170.98.47
rfc-editor.org.  1654 IN NS ns0.ietf.org.
rfc-editor.org.  1654 IN NS ns1.hkg1.afilias-nst.info.
...
;; ADDITIONAL SECTION:
ns0.ietf.org.     756   IN    A     64.170.98.2
ns0.ietf.org.     756   IN    AAAA  2001:1890:1112:1::14
```

In the command's output, the first two lines indicate the version of the `dig` program being used and the options provided to it, plus implied options (`+cmd` means that this information itself should be printed). The next portion indicates data in the DNS reply message: the `QUERY` opcode, `NOERROR` status indicating no errors were encountered, and a transaction ID of `53052`. In the *OpCode* field, `QUERY` is used for both queries and responses. Next, the `flags` line indicates that the message is a query response (`qr` flag) and not a query and that recursion was desired in the original query (`rd` flag) and is provided by the responding server (`ra` flag). The message contains a section with one query, and 12 resource records in the answer section (only 4 are shown). There are no RRs in the authority section, meaning that this response is likely from a caching server (the RRs are not authoritative). Different results might be obtained by interacting with different servers. The additional information section contains IPv4 and IPv6 addresses for one of the authoritative servers, should we wish to contact it. The question section contains a copy of our original query: type `ANY` for domain name `rfc-editor.org`.

Among the four RRs in the answer section shown, we find one `A` type, one `AAAA` type, and two `NS` types. From this information we can see that the domain name `rfc-editor.org` is a host with IPv4 address `64.170.98.47` and IPv6 address `2001:1890:1112:1::2f`. It is also a subdomain, as indicated by the presence of the NS records. We can quickly guess and verify that there is at least one host in this subdomain using the following command:

```
Linux% host ftp.rfc-editor.org
ftp.rfc-editor.org has address 64.170.98.47
```

This example indicates a few interesting aspects of A, AAAA, and NS records. First, it is possible for a single domain name to have records of each of these types (and more). This is fairly common for IPv6-capable servers that are the "well-known" servers for a particular organization. We can also see that each record has a TTL value, and they differ considerably, except for those in the same RRSet. The TTL for the records in the answer section is 1654s (about half an hour), and the TTL for records in the additional information section is 756s (about 12 minutes). Note that the TTL value of a cached record is never more than the TTL of the same record retrieved from the authoritative source. TTLs for cached records "decay" until the record is retrieved again from an authoritative server. As a result, retrieving a cached record multiple times from the same server usually shows a decreasing TTL value.

11.5.6.2 Example

Now that we have seen the DNS message format, transport protocol options, and RR types for basic queries and responses, let us see an example. We start with a simple case to see the communication between a resolver on a client, a local name server, and a remote name server managed by an ISP. This scenario demonstrates the importance of caching in DNS. The topology is shown in Figure 11-9.

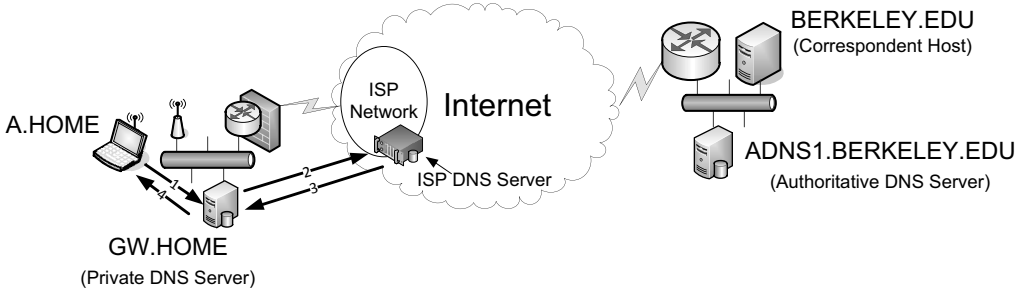

Figure 11-9 A simple DNS query/response example. The local DNS server (`GW.HOME`) provides recursion to the client (`A.HOME`), and uses the DNS server provided at the ISP when requested data is not present in the cache.

On our Windows client (`A.HOME`) we begin with a command that removes any DNS data cached by the resolver libraries. We then perform a query for the address (A record type) of the domain name `berkeley.edu`:

```
C:\> ipconfig /flushdns
Windows IP Configuration

Successfully flushed the DNS Resolver Cache.

C:\> nslookup
Default Server:  gw
Address:  10.0.0.1

> set type=a
> berkeley.edu.
Server:  gw
Address:  10.0.0.1

Non-authoritative answer:
Name:    berkeley.edu
Address:  169.229.131.81
```

The first command is specific to Windows and removes data cached by the client's resolver software. The `nslookup` program, available on both Windows and Linux/UNIX-based systems, provides a basic way to query the DNS for specific data. Upon execution, it indicates which name server it is using for resolution (here the server is `gw` at the address `10.0.0.1`). Using the `set` command, we arrange to query for A records, and then query for the name `berkeley.edu.`. Once again, `nslookup` indicates which server it uses for the resolution. It then also gives us an indication that the answer is nonauthoritative (i.e., it is being provided by a caching server) and the requested address is `169.229.131.81`.

To see what happens with the DNS protocol at the packet level, we use Wireshark and have a look at the first packet in detail, as shown in Figure 11-10.

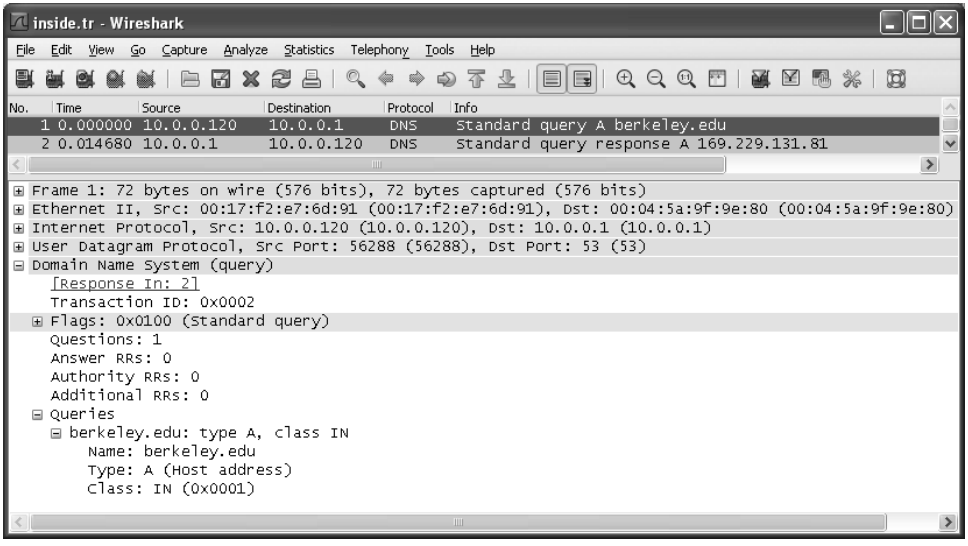

Figure 11-10 A UDP/IPv4 datagram containing a DNS standard query for the IPv4 address associated with `berkeley.edu`..

There are two messages in the trace: a standard query and a standard query response. In the first message (the query), the source IPv4 address is 10.0.0.120 (a DHCP-assigned address at the client; see Chapter 6), and the destination is 10.0.0.1 (the DNS server). The query is a UDP/IPv4 datagram with source port 56288 (an ephemeral port) and destination port 53 (the well-known DNS port). In terms of its full encapsulation, the request is an Ethernet frame containing 72 bytes. This size can be derived by summing the following parts: Ethernet header (14 bytes), IPv4 header (20 bytes), UDP header (8 bytes), DNS fixed header (12 bytes), query type (2 bytes), query class (2 bytes), plus the data labels for `berkeley` (9 bytes) and `edu` (4 bytes), plus the trailing 0 byte.

Turning to the details of the DNS header, the transaction ID is 0x0002 and forms the first 2 bytes of the DNS header, located at the start of the UDP payload. Only a single flag (recursion requested, the default) is set, so this message is a query. The message contains a standard query with one question. The other sections are empty. The question itself is for the name `berkeley.edu` and is seeking information of type A (address records) in the IN (Internet) class. After receiving this message, the name server process running on 10.0.0.1, unable to directly respond because it does not know the address, forwards the query to the next (upstream) name server it is configured to use. In this particular case, that name server is at the address 206.13.28.12 (see Figure 11-11).

In Figure 11-11 we see a query similar to the one sent by the client, but in this case the source IPv4 address is 70.231.136.162 (the ISP-side IPv4 address of `GW.HOME`). The destination address is 206.13.28.12, the IPv4 address of the ISP-provided DNS server, and the source port is an ephemeral port on the local DNS server (60961).

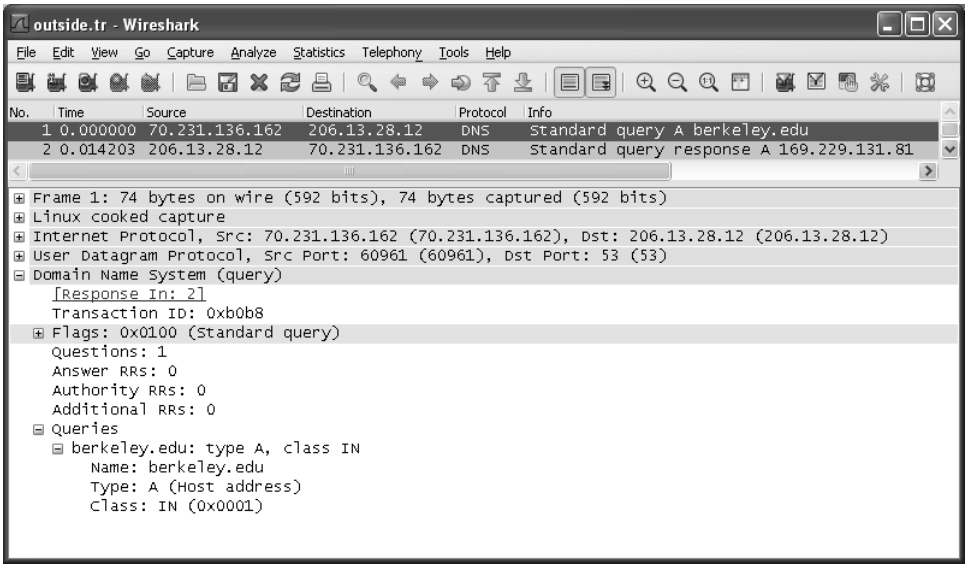

Figure 11-11 A DNS request generated at `GW.HOME` being sent to the ISP name server as a consequence of recursion.

The transaction ID is generated anew and set to 0xb0b8. Note that Wireshark indicates that the response to the query is contained in packet number 2.

Packet 2 in Figure 11-12 is the first DNS response we have seen. First, we note that the UDP source port number is 53, but the destination port is the ephemeral port number 60961. The transaction ID matches the query (0xb0b8), but the *Flags* field now contains the value 0x8180 (response, recursion requested, and recursion available are all set). The question section contains a copy of the question for which answers are being provided and typically matches the original query sent by the client exactly (e.g., case is preserved). There is one RR in the answer section. It is of type A (address), has a TTL of 10 minutes and a data length of 4 bytes (the size of an IPv4 address), and the value is 169.229.131.81, the IPv4 address we requested for `berkeley.edu`. Note that the authority flag is *not* set, and the authority section of the reply is empty. This response is based upon cached data; it is not authoritative for the domain. At this point, the local name server also caches the value (but only for up to 10 minutes as specified by the TTL in the RR it received) and responds to the requesting client (see Figure 11-13).

The response in Figure 11-13, packet 2, is much like the one from 206.13.28.12, except it is now sent from 10.0.0.1 to our original client at 10.0.0.120, and the transaction ID matches the one in the original DNS request. Note also that from the client's point of view the entire round-trip time of the transaction was about 14.7ms, but we know that most of that time (14.2ms) was taken up in the transaction between the local name server (`GW.HOME`) and the ISP's name server (206.13.28.12).

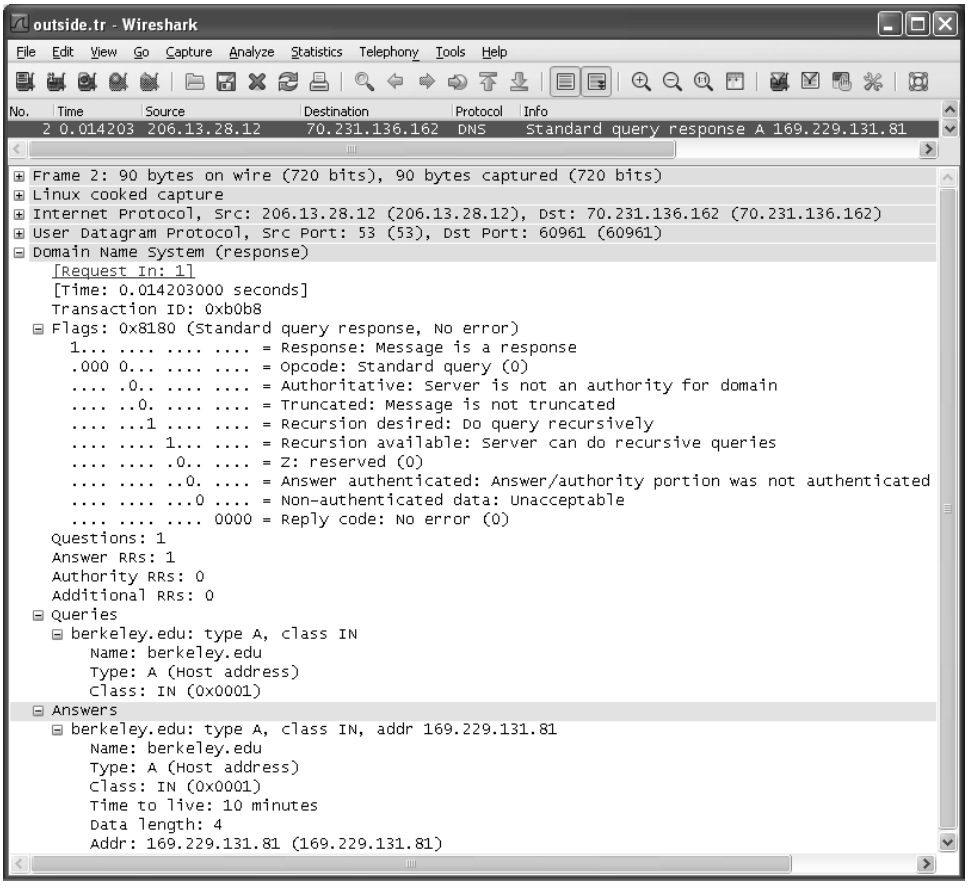

Figure 11-12 A standard DNS response sent from the ISP's DNS server back to `GW.HOME`.

11.5.6.3 Canonical Name (CNAME) Records

The CNAME record stands for *canonical name* record and is used to introduce an alias for a single domain name into the DNS naming system. For example, the name `www.berkeley.edu` may have a CNAME record that maps to some other machine (e.g., `www.w3.berkeley.edu`), so that if the Web server is located at a different computer, a relatively simple change to the DNS database may be all that is required for the rest of the world to find the new system. It is now common practice to use CNAME records to establish aliases for common services. As a result, names such as `www.berkeley.edu`, `ftp.sun.com`, `mail.berkeley.edu`, and `www.ucsd.edu` are all CNAME entries in the DNS that refer to other RRs.

Within a CNAME RR, the RDATA section contains the "canonical name" associated with the domain name (alias). Such names use the same type of encoding as other names (e.g., data labels and compression labels). When a CNAME RR is present for a particular name, no other data is permitted [RFC1912] (unless DNSSEC

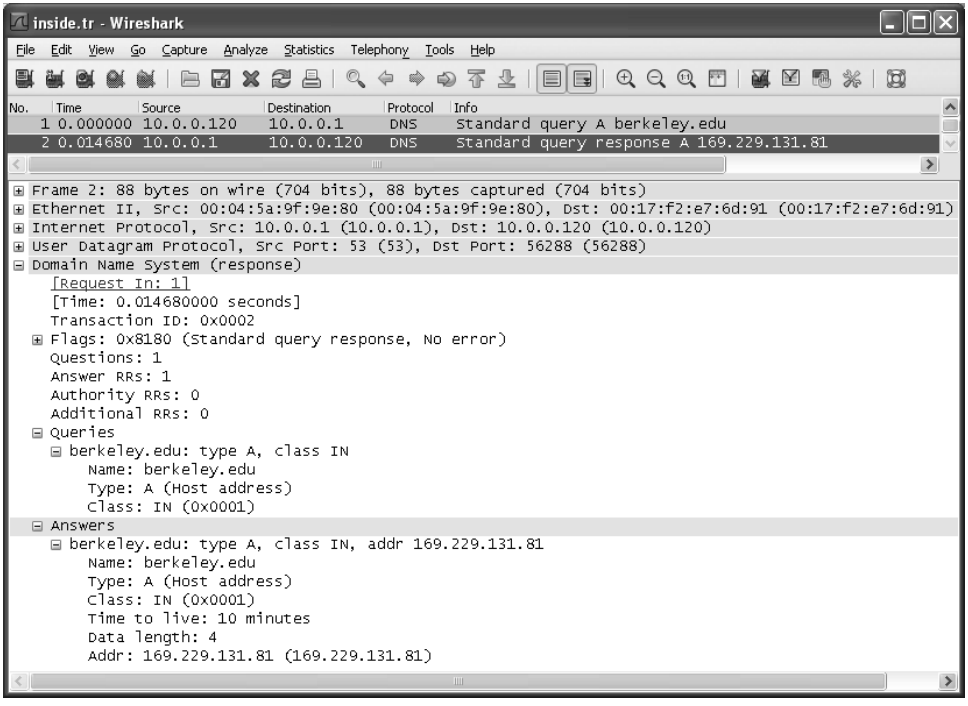

Figure 11-13 A response generated by `GW.HOME` and destined for the client. This message completes the recursive DNS transaction.

is in use; see Chapter 18). Domain names of CNAME RRs may not be used in all places that regular domain names can (e.g., as the target of an NS RR). Also, the canonical name may itself be a CNAME (called *CNAME chaining*), but this is usually discouraged, as it can cause DNS resolvers to make more queries than would otherwise be necessary. Nonetheless, there are certain services that make use of this feature. For example, the high-volume site `www.whitehouse.gov` (at the time of writing) uses a *content delivery network* (CDN)[2] provided by the Akamai Corporation. When we look up this domain name, we find the following:

```
Linux% host -t any www.whitehouse.gov
www.whitehouse.gov is an alias for www.whitehouse.gov.edgesuite.net.
Linux% host -t any www.whitehouse.gov.edgesuite.net
www.whitehouse.gov.edgesuite.net is an alias for a1128.h.akamai.net.
Linux% host -t any a1128.h.akamai.net
a1128.h.akamai.net has address 92.123.65.42
a1128.h.akamai.net has address 92.123.65.51
```

2. A content delivery network typically includes a number of synchronized content caches located in particular topological locations in the network. CDNs attempt to minimize latency for consumers accessing content in exchange for payment from content providers.

Thus, CNAME chains can be used with DNS. However, because of their potential performance impact, such chains are often limited by resolvers to a few "links" (such as five). Long chains are likely the result of an error in execution or a misunderstanding, as it is hard to imagine why they should be necessary under normal circumstances.

Note

There is a standard resource record called DNAME (type 39) [RFC2672][IDDN]. DNAME records act like CNAME records but for an entire zone. For example, all names of the form `NAME.example.com` could be mapped to `NAME.newexample.com` using a single DNAME resource record. However, DNAME records do not apply to the top-level record itself (`example.com` here).

11.5.6.4 Reverse DNS Queries: PTR (Pointer) Records

Although the most critical function of DNS is to provide mappings from names to IP addresses, there are many circumstances where the reverse mapping is required. For example, a server receiving an incoming TCP/IP connection request is able to ascertain the source IP address of the connection from the incoming IP datagram, but the name(s) corresponding to the address are not carried in the connection itself; such name(s) must be looked up in some other way. Fortunately, a clever use of the DNS can provide this capability.

The PTR RR type is used in response to reverse DNS queries, which are typically necessary when converting an IP address to a name. This uses the special `in-addr.arpa` (`ip6.arpa` for IPv6) domain, in a special way. Consider an IPv4 address such as 128.32.112.208. In the classful address structure (see Chapter 2), this address is taken from the 128.32 class B address space. To determine the name corresponding to this address, the address is first reversed, and then the special domain is added. In this example, a query for a PTR record using the name

```
208.112.32.128.in-addr.arpa.
```

would be used. In effect, this is a query for the "host" `208` in the "domain" `112.32.128.in-addr.arpa.`. We shall see more examples of reverse DNS queries later in this section.

Note

The regular DNS name space, which usually uses NS, A, and AAAA records, is not automatically linked with the "reverse" name space supported by PTR records. Thus it is possible (and even relatively common) to have an existing forward resolution that does not have a corresponding reverse mapping set up (or has a different one). Some services check to see that both directions are set up with equivalent mappings and may deny service under such circumstances.

Recall that IPv4 addresses are typically written in the "dotted-decimal" format and IPv6 addresses are written in the hex format (e.g., 169.229.131.81 and 2001:503:a83e::2:30, respectively). These addresses can be thought of as names existing in a left-to-right hierarchy. For example, the address 169.229.131.81 has the top-down hierarchy (reading left to right) 169, 229, 131, 81. By reversing the dotted-decimal IPv4 address and treating it as a DNS name, we can employ DNS to perform the mapping from IP address to name(s). So, the name 81.131.229.169 would effectively be the reversal of the IPv4 address 169.229.131.81. For IPv6, the scheme is similar, but any suppressed zeros are expanded, and each hexadecimal digit becomes a character. For example, the reversal of 2001:503:a83e::2:30 would be 0.3.0.0.2.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.e.3.8.a.3.0.5.0.1.0.0.2. Fortunately, users rarely have to type in these names directly.

As mentioned previously, the special domains `.in-addr.arpa` (for IPv4) and `.ip6.arpa` (for IPv6) are used in conjunction with the PTR ("pointer") RR type in support of these types of names and reverse DNS lookups. For example, consider the following commands:

```
C:\> nslookup
Default Server:  gw
Address:  10.0.0.1
> server c.in-addr-servers.arpa
Default Server:  c.in-addr-servers.arpa
Address:  196.216.169.10
> set type=ptr
> 81.131.229.169.in-addr.arpa.
Server:  c.in-addr-servers.arpa
Address: 196.216.169.10

169.in-addr.arpa  nameserver = w.arin.net
169.in-addr.arpa  nameserver = t.arin.net
169.in-addr.arpa  nameserver = dill.arin.net
169.in-addr.arpa  nameserver = x.arin.net
169.in-addr.arpa  nameserver = z.arin.net
169.in-addr.arpa  nameserver = y.arin.net
169.in-addr.arpa  nameserver = u.arin.net
169.in-addr.arpa  nameserver = v.arin.net
```

This example shows how the `.in-addr.arpa` domain is set up. According to [RFC5855], the `in-addr-servers.arpa` and `ip6-servers.arpa` domains are used in forming the domain names associated with the servers that provide reverse DNS mappings for IPv4 and IPv6, respectively. As of 2011, there are five such servers for each version of IP: *X*`.in-addr-servers.arpa` and *X*`.ip6-servers.arpa`, where *X* is any letter `a` through `f` (inclusive).

Although the ten servers we have mentioned contain authoritative data for reverse mappings, they do not contain the information we are looking for. In our example, the first server contacted instead told us to contact one of the eight name servers maintained by ARIN, the American Registry for Internet Numbers, which is authoritative for IPv4 addresses that start with 169. If we in turn contact one

of these servers, we find that a PTR query for `81.131.229.169.in-addr.arpa.` gives the following response:

```
> server w.arin.net
Default Server: w.arin.net
Address: 72.52.71. 2
Default Server: w.arin.net
Address: 2001:470:1a::2
> 81.131.229.169.in-addr.arpa.
Server:  w.arin.net
Address: 72.52.71.2

229.169.in-addr.arpa nameserver = adns1.berkeley.edu.
229.169.in-addr.arpa nameserver = phloem.uoregon.edu.
229.169.in-addr.arpa nameserver = aodns1.berkeley.edu.
229.169.in-addr.arpa nameserver = adns2.berkeley.edu.
```

Here we can surmise that the network prefix `169.229/16` is owned by an educational institution called Berkeley, that the campus maintains three name servers covering its `in-addr.arpa` space, and that the University of Oregon also provides a copy. Continuing by contacting one of these servers, we find our answer (this time using the Linux version of `nslookup` with slightly different output):

```
Linux% nslookup
> set type=ptr
> server adns1.berkeley.edu
Default Server:  adns1.berkeley.edu
Address:  128.32.136.3#53
Default Server:  adns1.berkeley.edu
Address:  2607:f140:ffff:fffe::3#53
> 81.131.229.169.in-addr.arpa.
Server:  adns1.berkeley.edu
Address: 128.32.136.3#53

81.131.229.169.in-addr.arpa     name = webfarm.Berkeley.EDU
```

Here we obtain the result we were looking for, that the IPv4 address `169.229.131.81` has the name `webfarm.Berkeley.EDU`. The DNS server uses port 53, as indicated by the `#53` following the IP addresses. This output makes it obvious that accessing the DNS with UDP/IPv4 (as opposed to UDP/IPv6) can still provide mappings for IPv6 addresses using "quad-A" (AAAA) DNS records because we can see that the IPv6 address of the server is `2607:f140:ffff:fffe::3`.

If there were not a separate branch of the DNS tree for handling the address-to-name translation, there would be essentially no way to do the reverse translation other than starting at the root of the tree and trying every top-level domain. This is clearly an unreasonable option, given the current size of the Internet. The `in-addr.arpa` solution is effective and fairly efficient, although the reversed bytes of the IPv4/IPv6 address and the special domains can be confusing.

Fortunately, as mentioned before, users can typically avoid having to type or refer to them. Even application writers do not typically have to manipulate addresses to perform reverse queries, as library functions (such as the C library function `getnameinfo()`) perform this task.

It is worth mentioning here that PTR queries have become a significant concern for the global DNS servers. Consider a home network using one of the private address prefixes such as 10.0.0.0/8 (IPv4) or fc00:/7 (IPv6). When a system receives an incoming connection request from another system on the same privately addressed subnet, it may wish to resolve the source address to a name and does so by performing a PTR query. If the query is not answered by the local DNS server, it will likely propagate to the global Internet. For this reason (and a few others), [RFC6303] specifies that local name servers—especially those operating in networks using private IP addressing that are attached to the Internet—provide PTR mappings for the private address space defined in [RFC1918] for IPv4 and [RFC4193] for IPv6 (i.e., in IN-ADDR.ARPA and D.F.IP6.ARPA, respectively).

11.5.6.5 Classless `in-addr.arpa` *Delegation*

When organizations join the Internet and obtain authority to fill in a portion of the DNS name space, they often also obtain authority for a portion of the `in-addr.arpa` name space corresponding to their IPv4 addresses on the Internet. In the case of UC Berkeley, authority includes the network prefix 169.229/16, which, using older terminology, is "class B" network number 169.229. Thus, UC Berkeley would be expected to populate a portion of the DNS tree with PTR records using names ending in `229.169.in-addr.arpa`. This works fine for cases where the address prefix assigned to the organization is one of the older class A, B, or C styles where the number of bits is an integral multiple of 8. However, many organizations today are given prefix lengths of greater than 24 bits or greater than 16 bits (but less than 24). In these cases, the address range is not easily written as a simple reversal of the IP address. Instead, some method of conveying the network prefix length must be included as well.

The standard method for implementing this, given by [RFC2317], is to append the length of the prefix to the reversed octets and use it as the first label in the domain name. For example, assume that a site is assigned the prefix 12.17.136.128/25, a prefix that includes 128 addresses. According to [RFC2317], two types of records should be provided. First, for each name of the form *X*`.136.17.12.in-addr.arpa` (where *X* is at least 128 and not more than 255), a CNAME RR is created, likely maintained by a site's ISP, according to the following pattern:

```
128.136.17.12.in-addr.arpa. canonical name =
                            128.128/25.136.17.12.in-addr.arpa.
129.136.17.12.in-addr.arpa. canonical name =
                            129.128/25.136.17.12.in-addr.arpa.
...
255.136.17.12.in-addr.arpa. canonical name =
                            255.128/25.136.17.12.in-addr.arpa.
```

Here we can see how the network prefix is encoded, with the / notation associated with the second label in the domain name (for this example). These entries are typically placed by an ISP and allow for delegations on non-byte-aligned address ranges. In this example, the customer is now able to provide mappings for the zone 128.128/25.136.17.12.in-addr.arpa. We can trace the delegation as follows:

```
C:\> nslookup
Default Server:  gw
Address:  10.0.0.1
> server f.in-addr-servers.arpa
Default Server:  f.in-addr-servers.arpa
Addresses:  193.0.9.1
> set type=ptr
> 129.128/25.136.17.12.in-addr.arpa.
Server:  f.in-addr-servers.arpa
Address:  193.0.9.1
12.in-addr.arpa nameserver = dbru.br.ns.els-gms.att.net
12.in-addr.arpa nameserver = cbru.br.ns.els-gms.att.net
12.in-addr.arpa nameserver = cmtu.mt.ns.els-gms.att.net
12.in-addr.arpa nameserver = dmtu.mt.ns.els-gms.att.net
> server dbru.br.ns.els-gms.att.net.
Default Server:  dbru.br.ns.els-gms.att.net
Address:  199.191.128.106

> 129.128/25.136.17.12.in-addr.arpa.
128/25.136.17.12.in-addr.arpa   nameserver = ns2.intel-research.net
128/25.136.17.12.in-addr.arpa   nameserver= ns1.intel-research.net

> server ns1.intel-research.net.
Server:  ns1.intel-research.net
Address:  12.155.161.131
> 129.128/25.136.17.12.in-addr.arpa.

129.128/25.136.17.12.in-addr.arpa
                    name = dmz.slouter.seattle.intel-research.net
128/25.136.17.12.in-addr.arpa
              nameserver = bldmzsvr.berkeley.intel-research.net
128/25.136.17.12.in-addr.arpa
              nameserver = sldmzsvr.intel-research.net
bldmzsvr.berkeley.intel-research.net internet address = 12.155.161.131
sldmzsvr.intel-research.net internet address = 12.17.136.131
```

In this example, we wish to find out the name for the host associated with IPv4 address 12.17.136.129. We have already seen that it has a CNAME RR pointing to the canonical name 129.128/25.136.17.12.in-addr.arpa.. We instruct our resolver to use one of the root servers (F) and arrange for the query type to be for a PTR RR. At this point we request a resolution for 129.128/25.136.17.12.in-addr.arpa.. The root name server does not have this information, and it does not perform recursion, so it returns the name of the authoritative servers for the domain

12.in-addr.arpa.. Picking one of them (DBRU), we again try to resolve our question. This time we find two name servers (ns1 and ns2). Picking one of these, we are able to resolve the PTR request. It resolves to the name dmz.slouter.seattle.intel-research.net.

11.5.6.6 Authority (SOA) Records

In DNS, each zone has an authority record, using an RR type called *start of authority* (SOA). These records provide authoritative links between portions of the DNS name space and the servers that provide the zone information allowing various queries to be performed for addresses and other information. The SOA RR is used to identify the name of the host providing the official permanent database, the responsible party's e-mail address (where "." is used instead of @), zone update parameters, and the default TTL. The default TTL is applied to RRs in the zone that are not otherwise assigned an explicit per-RR TTL.

The zone update parameters include a serial number, refresh time, retry time, and expire time. The serial number is increased (by at least 1), usually by the network administrator, anytime there is a change to the zone contents. It is used by secondary servers to determine if they should initiate a zone transfer (when they do not have a copy of the zone contents with largest serial number). The refresh time tells secondary servers how long to wait before checking the SOA record from the primary and its version number to determine if a zone transfer is required. The retry and expire times are used in the case of zone transfer failure. The retry value gives the time (in seconds) a secondary will wait before retrying. The expire time is an upper bound (in seconds) that a secondary server will keep retrying zone transfers before giving up. If it gives up, such a server ceases to respond to queries for the zone. In general, a zone can contain a mix of IPv4 and IPv6 data and can be accessed using either version of IP. In this example, we use IPv6 (using nslookup on an IPv6-only Windows host):

```
C:\> nslookup
Default Server:  gw
Address:  fe80::204:5aff:fe9f:9e80

> set type=soa
> berkeley.edu.
Server:  gw
Address:  fe80::204:5aff:fe9f:9e80

Non-authoritative answer:
berkeley.edu
        primary name server = ns-master1.berkeley.edu
        responsible mail addr = hostmaster.berkeley.edu
        serial  = 2009050116
        refresh = 10800 (3 hours)
        retry   = 1800 (30 mins)
        expire  = 3600000 (41 days 16 hours)
        default TTL = 300 (5 mins)
```

```
> server adns1.berkeley.edu.
Default Server:  adns1.berkeley.edu
Addresses:  2607:f140:ffff:fffe::3
          128.32.136.3

> berkeley.edu.
Server:  adns1.berkeley.edu
Addresses:  2607:f140:ffff:fffe::3
          128.32.136.3

berkeley.edu
        primary name server = ns-master1.berkeley.edu
        responsible mail addr = hostmaster.berkeley.edu
        serial  = 2009050116
        refresh = 10800 (3 hours)
        retry   = 1800 (30 mins)
        expire  = 3600000 (41 days 16 hours)
        default TTL = 300 (5 mins)
berkeley.edu    nameserver = ns.v6.berkeley.edu
berkeley.edu    nameserver = aodns1.berkeley.edu
berkeley.edu    nameserver = adns2.berkeley.edu
berkeley.edu    nameserver = phloem.uoregon.edu
berkeley.edu    nameserver = adns1.berkeley.edu
berkeley.edu    nameserver = ucb-ns.NYU.edu
ns.v6.berkeley.edu      internet address = 128.32.136.6
ns.v6.berkeley.edu      AAAA IPv6 address = 2607:f140:ffff:fffe::6
adns1.berkeley.edu      internet address = 128.32.136.3
adns1.berkeley.edu      AAAA IPv6 address = 2607:f140:ffff:fffe::3
adns2.berkeley.edu      internet address = 128.32.136.14
adns2.berkeley.edu      AAAA IPv6 address = 2607:f140:ffff:fffe::e
aodns1.berkeley.edu     internet address = 192.35.225.133
aodns1.berkeley.edu     AAAA IPv6 address =
                                2607:f010:3f8:8000:214:4fff:fe45:e6a2
phloem.uoregon.edu      internet address = 128.223.32.35
phloem.uoregon.edu      AAAA IPv6 address = 2001:468:d01:20::80df:2023
```

Here we can see that not only did we receive the SOA record, but we also received a list of six authoritative name servers, and the IPv4/IPv6 addresses (glue records) for five of them (the address for the NYU server is not given, as glue records for `NYU.edu` would be in a different zone supported by a different server). As this is one of the more interesting responses we have seen, let us look at the packet contents corresponding to the request sent to the authoritative name server, `adns1.berkeley.edu` (see Figure 11-14).

This trace contains two packets, and we have chosen to display the reply, which is the more interesting of the two. A query for an SOA RR was sent to the host 2607:f140:ffff:fffe::3 (`adns1.Berkeley.EDU`) from the local system's globally scoped IPv6 address 2001:5c0:1101:ed00:5571:5f81:e0a6:4978. The response is carried in an IPv6 datagram with 491 bytes total length (the *Payload Length* field is 451). This particular packet contains the IPv6 header (40 bytes), UDP header (8 bytes), plus the DNS message (443 bytes). The DNS message includes one question, one answer, six authority RRs, and ten additional RRs.

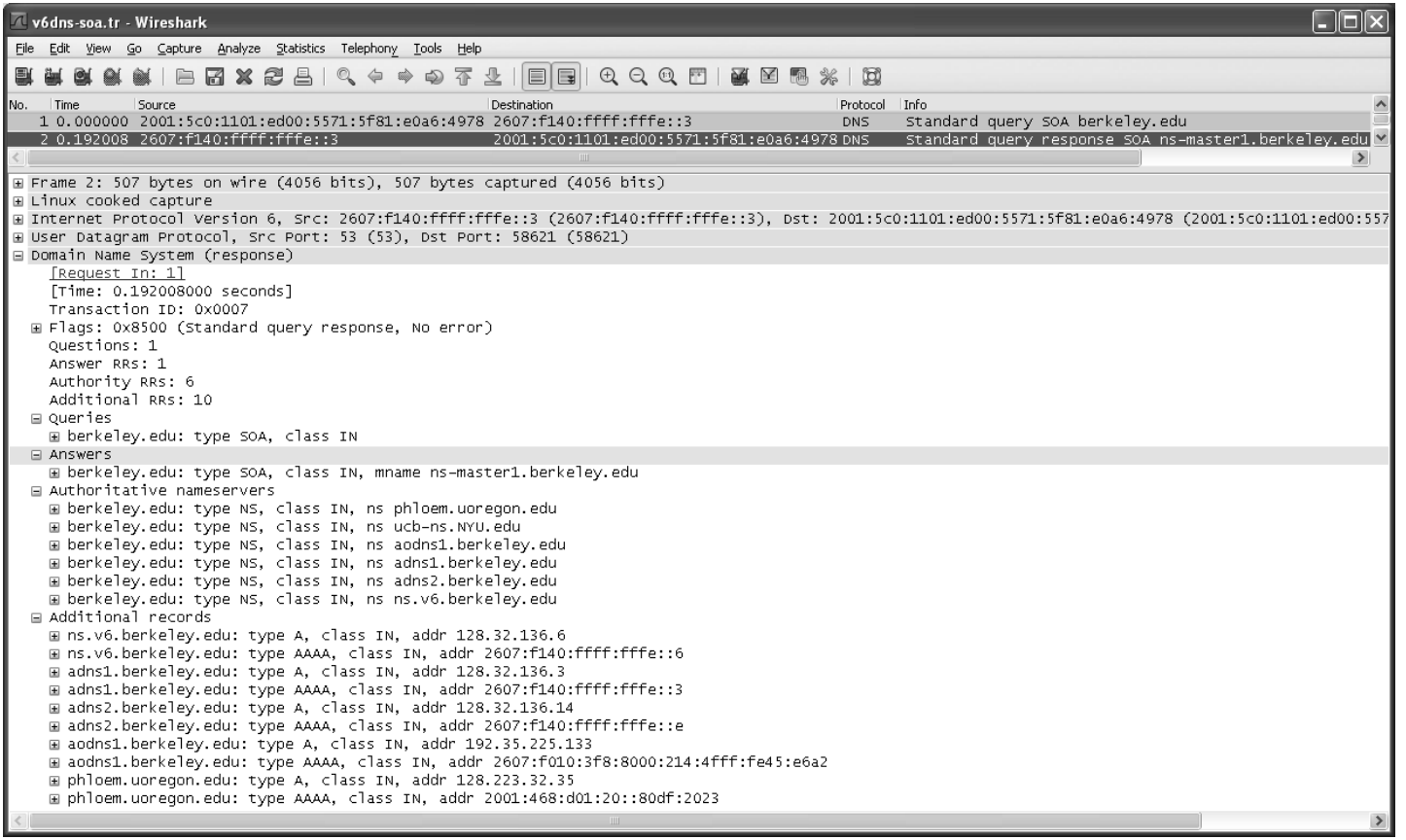

Figure 11-14 Response to a DNS query for an SOA record using IPv6. The response includes IPv4 and IPv6 addresses for the zone.

The question section contains the labels `berkeley` and `edu` and is 18 bytes long. The answer section contains the relevant information for the `berkeley.edu` domain described earlier and is able to take advantage of compression labels thanks to the contents of the question section. The total length for this section is 58 bytes. The authority section contains six NS records identifying name servers. This information takes another 135 bytes. The additional information section includes five A records and five AAAA records for a total of 220 bytes. The size of the *RDATA* field for each AAAA record is 16 bytes, so although the IPv6 address can be written in textual form with the `::` convention to save space, it is not encoded this way in the packet. Instead, the full 128 bits of the address are used.

11.5.6.7 Mail Exchanger (MX) Records

An MX record provides the name of a *mail exchanger*—a host willing to engage in the *Simple Mail Transfer Protocol* (SMTP) [RFC5321] to receive incoming e-mail on behalf of users associated with a domain name. When the Internet was still developing, some sites did not have permanent connections but instead would dial up and connect to hosts that did have permanent Internet connections. In such scenarios, the e-mail destination might be disconnected from the network when e-mail was in transit, so another host would hold on to the mail until the destination was attached. This was one motivation for the inclusion of MX records in the DNS—to allow sending hosts to deliver e-mail to an intermediary ("relay server") even if the true destination was not available. Today, MX records are still used, and mail agents prefer to deliver e-mail to the host(s) listed in an MX record associated with a particular domain name.

MX records include a *preference* value, so that more than one MX record may be present for a particular domain name. The preference value allows a sending agent to sort the hosts in preference order (smaller is more preferable) in deciding which host to use as an e-mail destination. For example, we can use the `host` command again to query the DNS for MX records associated with the domain name `cs.ucla.edu`:

```
Linux% host -t MX cs.ucla.edu ns3.dns.ucla.edu
Using domain server:
Name: ns3.dns.ucla.edu
Address: 2607:f600:8001:1::ff:fe01:35#53
Aliases:

cs.ucla.edu mail is handled by 13 Pelican.cs.ucla.edu.
cs.ucla.edu mail is handled by 3 Moa.cs.ucla.edu.
cs.ucla.edu mail is handled by 13 Mailman.cs.ucla.edu.
```

Here we can see that an e-mail addressed to `person@cs.ucla.edu` is handled by one of three mail servers configured in the DNS. All of these mail servers are part of the `cs.ucla.edu` domain, but in general mail servers do not have to be named with the same domain as the e-mail they are handling. These three servers can be grouped into two parts: one with preference 3 and one set with preference

13. The server with the smaller preference number is preferred, so the sender first tries `Moa.cs.ucla.edu`. If that fails, it tries either `Pelican` or `Mailman`, selected at random.

It is possible that none of the MX record target hosts is reachable. This is an error condition. It is also possible that there are no MX records present, but there are CNAME, A, or AAAA records for the domain name. If there is a CNAME record, the target of the CNAME is used in place of the original domain name. If there are A or AAAA records, the mail agent may connect to these addresses. Each is considered to have a preference of zero (called *implicit MX*). MX record targets must be domain names that resolve to A or AAAA records; they cannot point to CNAMEs [RFC5321].

11.5.6.8 Fighting Spam: The Sender Policy Framework (SPF) and Text (TXT) Records

For outgoing e-mail, MX records allow the DNS to help determine the names of mail relays and servers for a domain. More recently, the DNS has been leveraged by receiving mail agents to determine which relaying or sending mail servers are authorized to send mail from a particular domain name. This is used to help combat spam (unwanted e-mail) that is sent by a rogue mail agent pretending to be an authorized mail sender.

E-mail received by a mail server is rejected, stored, or forwarded to another mail server. Rejection can happen for a number of reasons, such as a protocol error or lack of available storage space at the receiver. It can also be rejected because the sending mail client does not appear to be the proper one for sending e-mail. This capability is supported by the *Sender Policy Framework* (SPF) and documented in [RFC4408], an experimental RFC. There is another framework known as *Sender ID* [RFC4406] that incorporates SPF's functions. It is also experimental but less widely deployed.

Version 1 of SPF uses DNS TXT or SPF (type 99) resource records. Records are set up and published in the DNS by a domain's owner to indicate which servers are authorized to send mail originating from the domain. Although the SPF record type is a more "proper" place to carry SPF-related information in some sense, some DNS client implementations do not process SPF records properly, so to avoid this complication TXT records are used. TXT records hold simple strings associated with a domain name. Historically they have held strings useful for human consumption, to aid in debugging or identifying the owner or location of a domain. Today, they are usually processed by programs such as the SPF application.

SPF supports a rich syntax to express criteria used to match against details about an incoming mail message and the connection in which it is carried. For example, UC Berkeley uses the following SPF entry (some lines have been wrapped for clarity):

```
Linux% host -t txt berkeley.edu
berkeley.edu descriptive text
       "v=spf1 ip4:169.229.218.128/25 ip6:2607:F140:0:1000::/64
       include:outboundmail.convio.net ~all"
```

In this example, the information being provided is for SPF version 1 (indicated by the `v=spf1` string in the version section) and uses a TXT RR. When a receiving mail agent receives e-mail purportedly coming from the domain `berkeley.edu`, it performs a DNS query for records of type TXT against the `berkeley.edu` domain. The value of the text record contains the matching criteria (called *mechanisms*) and other information (called *modifiers*). Preceding each mechanism is a *qualifier* that determines the consequence of a matching mechanism. Processing of SPF records takes place using a function called `check_host()`. The function evaluates various mechanisms and completes when the first matching mechanism is encountered. Ultimately, `check_host()` provides a return value that is one of the following: None, Neutral, Pass, Fail, SoftFail, TempError, PermError. The None and Neutral return values indicate that no information was available or that information was available but that no result is asserted. These are handled identically. Pass indicates a match, as described in the next paragraph. Fail indicates that the sending host is not authorized to send mail from the domain. SoftFail is somewhat ambiguous but is to be treated "somewhere between a 'Fail' and a 'Neutral,'" according to [RFC4408]. The TempError return indicates some transient failure (e.g., communication failure) that is likely to abate. The PermError return indicates that there was a problem in the SPF configuration, usually due to a malformed TXT or SPF record for the domain.

Reading from left to right in the example, the string `v=spf1` is a modifier indicating that the SPF version is 1. The `ip4` mechanism specifies that the SMTP sender has an IPv4 address from the prefix `169.229.218.128/25`. The `ip6` mechanism specifies any sending host with IPv6 address prefix `2607:F140:0:1000::/64`. Finally, the `include` mechanism incorporates, by reference, the TXT records with `outboundmail.convio.net`:

```
Linux% host –t txt outboundmail.convio.net
outboundmail.convio.net descriptive text
       "v=spf1 +ip4:66.45.103.0/25 +ip4:69.48.252.128/25
       +ip4:209.163.168.192/26 ~all"
outboundmail.convio.net descriptive text
       "spf2.0/pra
       +ip4:66.45.103.0/25 +ip4:69.48.252.128/25
       +ip4:209.163.168.192/26 ~all"
```

Note that these TXT records are used for both SPF and for Sender ID (which uses the value of `spf2.0/pra` in the version section). The first record is used by SPF. The `+` qualifier indicates that a match results in a Pass indication. Any mechanism missing a qualifier is assumed to have the `+` qualifier. Other possible qualifiers include `–` (Fail), `~` (Soft Fail), and `?` (Neutral). If none of the matching mechanisms produces a Pass result, the final mechanism (`all`) matches any condition. The tilde character (~) before the `all` criterion indicates that a SoftFail return should be generated if `all` is the only matching mechanism. The exact way a soft failure is handled is dependent on the receiving e-mail software. Note that

even with SPF support, validation is provided only on the sending domain and system, and not on the sending user. In Chapter 18 we will look at DKIM, which provides SPF-like capabilities but uses cryptography for authentication.

11.5.6.9 Option (OPT) Pseudo-Records

In conjunction with EDNS0, described previously, a special *OPT pseudo-RR* has been defined [RFC2671]. It is "pseudo" in the sense that it pertains only to the contents of a single DNS message and is not conventional DNS RR data. Consequently, OPT RRs are not cached, forwarded, or persistently stored, and they may appear only once (or not at all) in a DNS message. If one is present in a DNS message, it is found in the additional information section.

An OPT RR contains a 10-byte fixed portion followed by a variable portion. The fixed portion includes 16 bits indicating the RR type (41), 16 bits indicating the UDP payload size, 32 bits constituting an extended *RCODE* field and flags area, and 16 bits giving the size of the variable portion in bytes. These fields are located in the same relative positions as the *Name*, *Type*, *Class*, *TTL*, and *RDLEN* fields, respectively, in a conventional RR (see Figure 11-8). OPT RRs use a null domain name in the *Name* field (0 bytes). The extended *RCODE* and *Flags* area (32 bits, corresponding to the *TTL* field in Figure 11-8) is subdivided into an 8-bit area to hold an extra 8 high-order bits augmenting the *RCODE* field in Figure 11-3, and an 8-bit *Version* field (currently set to 0 to indicate EDNS0). The remaining 16 bits are not yet defined and must be 0. The additional 8 bits provide an extended set of possible DNS error types, and these values are given in Table 11-4. (Note that value 16 is defined by two distinct RFCs.)

Table 11-4 Extended *RCODE* values. Most are used to support security extensions.

Value	Name	Reference	Description and Purpose
16	BADVERS	[RFC2671]	Bad EDNS version
16	BADSIG	[RFC2845]	Bad TSIG signature (see Chapter 18)
17	BADKEY	[RFC2845]	Bad TSIG key (see Chapter 18)
18	BADTIME	[RFC2845]	Bad TSIG signature (time problem; see Chapter 18)
19	BADMODE	[RFC2930]	Bad TKEY mode (see Chapter 18)
20	BADNAME	[RFC2930]	Duplicate key name (see Chapter 18)
21	BADALG	[RFC2930]	Algorithm not supported (see Chapter 18)

As we have mentioned, OPT RRs contain a variable-length *RDATA* field. This field is used to hold an extensible list of attribute-value pairs. The current set of attributes, meanings, and defining RFCs is maintained by the IANA [DNSPARAMS]. One such option, called NSID (EDNS option code 3) [RFC5001], indicates a special identifying value for a responding DNS server. The format of this value is not defined by standard but is instead configurable by the system administrator

of the DNS server. This capability may be useful in circumstances where an anycast address is used to identify a group of servers. The NSID is able to identify a specific responding server using a value other than the sending IP address. We shall see more examples of OPT RRs and EDNS0 usage when we look at DNSSEC in Chapter 18.

11.5.6.10 Service (SRV) Records

[RFC2782] defines the *service* (SRV) resource record. SRV RRs generalize the MX record format to describe the host, protocols, and port numbers used to contact a particular service. An SRV RR is ordinarily structured as follows:

```
_Service._Proto.Name TTL IN  SRV    Prio    Weight    Port    Target
```

The *Service* identifier is the official name of a service. The *Proto* identifier is the transport protocol used to access the service, usually TCP or UDP. The TTL value is a conventional RR TTL, and IN and SRV indicate the Internet class and SRV RR type, respectively. The *Prio* value is a 16-bit unsigned value and works like the priority value in MX records (lower numbers represent higher priorities). The *Weight* value is used to choose an RR among several whose priority values are equal. The idea is that the weight is to be used as a weighted probability to select the particular entry for load balancing, so larger weights indicate a greater probability of selection. The *Port* is the TCP or UDP (or other transport protocol's) port number. The *Target* is the domain name of the target host where the service is being provided. The *Name* identifier is the containing domain in which a particular service is to be found. One of the purposes of SRV records is to identify when multiple individual servers in a domain support the same service.

For example, if a client would like to determine the host and port where the `ldap` service is available using the TCP protocol in the domain `example.com`, it would perform a query for SRV records using the domain name `_ldap._tcp.example.com`. Here is a real-world example:

```
Linux% host -t srv _ldap._tcp.openldap.org
_ldap._tcp.openldap.org has SRV record 0 0 389 www.openldap.org.
```

In this example, we are looking for a server providing the *Lightweight Directory Access Protocol* (LDAP) [RFC4510] service over TCP within the domain `openldap.org`. We find that it can be accessed at the server `www.openldap.org` using TCP port `389` (the default LDAP port). The *Priority* and *Weight* values are `0`, as there are no alternative servers.

[RFC2782] did not specify a new IANA registry for SRV *Service* and *Proto* values. So, by default, the names correspond to the names maintained in IANA's "Service Name and Transport Protocol Port Number" registry [ISPR], and the *Proto* values are either `_tcp` or `_udp`. There are a few exceptions, however. [RFC5509] establishes conventions for SIP-based presence and instant messaging using the

following SRV *Service* and *Proto* names: `_im._sip` and `_pres._sip`. [RFC6186] defines the following SRV *Service* names for e-mail user agents to easily discover the contact information for IMAPS, SMTP, IMAP, and POP3 servers (the first two are ordinarily preferred when setting up an an e-mail client): `_submission`, `_imap`, `_imaps`, `_pop3`, `_pop3s`. Although [RFC6186] doesn't require these names to use TCP as the corresponding *Proto* value, this is currently the only real option. For example, a user configuring a new *mail user agent* (MUA, essentially an e-mail program) might specify only the domain `example.com`. The MUA implementation would then likely perform DNS queries for at least `_submission._tcp.example.com` and `_imaps._tcp.example.com`.

11.5.6.11 Name Authority Pointer (NAPTR) Records

The *Name Authority Pointer* (NAPTR) RR type is used when DNS supports a *Dynamic Delegation Discovery System* (DDDS) [RFC3401]. A DDDS is a general, abstract algorithm for applying dynamically retrieved string transformation rules to strings provided by applications and using the results, most often, for locating resources. Each DDDS application customizes the operation of the general DDDS rules for its particular use case. A DDDS includes a rules database and a set of algorithms for forming strings that are used with the database to produce output strings. DNS is one such database [RFC3403], and with it the NAPTR resource record type is used to hold the transformation rules. One such DDDS application has been defined for use with DNS to handle multinational telephone numbers and convert them to a standard *Uniform Resource Identifier* (URI) format [RFC3986] using ENUM (see Section 11.5.6.12).

In a DDDS, an *algorithm* [RFC3402] directs how an *application-unique string* (AUS) is processed by rules contained in a database. The result can be either a *terminal string* (complete output) or another (nonterminal) string used to retrieve another rule that is applied to the AUS. In all, the collection forms a powerful string rewriting system that can be used to encode nearly anything that has a sufficiently regular syntax. The essence of this algorithm is captured in Figure 11-15.

The process illustrated in Figure 11-15 starts by applying the first Well-Known Rule to the AUS, which is uniquely identified for each application. The result forms a key used to retrieve another rule from a database. Rules are string-rewriting patterns and flags that are applied to the AUS, but never to the result of a rewritten string. The particular way this works is dependent on the application, but usually the rules are regular expression substitutions, similar to those used with the UNIX `sed` program [DR97]. When using the DNS as a database for supporting a DDDS [RFC3403], the case in which we are interested, the keys are domain names and the rules are stored in NAPTR resource records. Each NAPTR RR contains the following fields: *Order*, *Preference*, *Flags*, *Services*, *Regular Expression* (sometimes abbreviated *Regexp*), and *Replacement*.

The *Order* field is a 16-bit unsigned integer specifying which NAPTR record to use before others (lower numbers are preferred to higher ones), as the DNS architecture does not guarantee the ordering of any particular set of resource records.

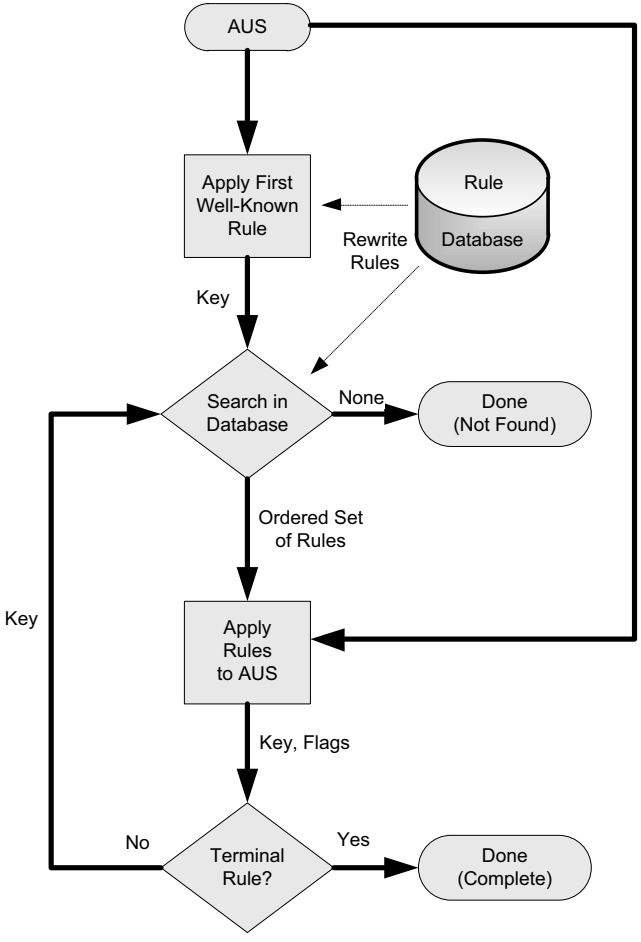

Figure 11-15 Abstract operation of the DDDS algorithm. Non-terminal records are permitted to form loops. Each iteration involves a string rewrite operation on the application's unique string.

The *Preference* field is used to influence the order of records containing the same order number. The *Order* field is supposed to place a mandatory ordering on RRs, whereas the preference number is advisory. The *Flags* field contains an unordered list of single characters from the set A–Z and 0–9 (case-insensitive). The particular application using NAPTR records (e.g., ENUM, described in the next section) defines the interpretation of the *Flags* field. The *Services* field is defined by the application to indicate which type of service is being described. The *Regular Expression* field contains a substitution expression that is applied to the AUS to form the identity of another server to use for another NAPTR lookup (non-terminal case) or the output string (terminal case). The *Replacement* field (which exists only when the *Regular Expression* does not) indicates the next server to query

for NAPTR records. It is encoded as a separate FQDN (no name compression is used within the DNS message). The uses for these two final (mutually exclusive) fields are very similar for historical reasons in the development of the NAPTR RR.

To get a better sense of how NAPTR processing works with applications, we will have a brief look at the ENUM and SIP DDDS applications, the URI/URN DDDS applications, and alternatives for regular NAPTR records called S-NAPTR and U-NAPTR. Specifying a DDDS entails specifying the application's AUS, first Well-Known Rule, expected output, valid databases, flags, and service parameters.

11.5.6.12 ENUM and SIP

In the ENUM DDDS [R06][RFC6116][RFC6117][RFC5483], which is used to map phone numbers to URI information, the AUS is an E.164-format telephone number (up to 15 digits starting with the + character). The initial + character differentiates E.164 numbers acceptable for use with the ENUM DDS from numbers in other name spaces. The first Well-Known Rule starts by removing any dashes or other non-digit characters in the AUS. The DDDS database is the DNS, where keys are domain names created from the AUS (which now consists only of digits) as follows: dot (`.`) characters are inserted between each digit and the result is reversed. Then, the suffix `.e164.arpa` is added. For example, the E.164 number `+1-415-555-1212` would be tranformed to the key `2.1.2.1.5.5.5.1.4.1.e164.arpa`. The resulting domain name is used to query for NAPTR records.

The final output, possibly after multiple loops of the DDDS algorithm shown in Figure 11-15, is an absolute (not relative) URI. The only flag defined is the *U* flag, indicating a terminal rule that produces a URI. The lack of any flag indicates a non-terminal rule, sometimes called a *non-terminal NAPTR* (NTN). The service parameters, encoded in the *Service* field of the NAPTR record, are of the form E2U+*`Service`*, which derives from the string E2U (an indicator for E.164 to URI) plus a *`Service`* name subfield providing information about particular services associated with the number. Together, they form an *enumservice* identifier, and such services are registered with the IANA [ENUM][RFC6117]. Many have been created, including enumservices for fax, instant messaging, and presence indicators.

To see how this all works, we can construct a query for the number `+420738511111` at the University of Ostrava in the Czech Republic (lines are wrapped for clarity):

```
Linux% host -t naptr 1.1.1.1.1.5.8.3.7.0.2.4.e164.arpa
1.1.1.1.1.5.8.3.7.0.2.4.e164.arpa has NAPTR record
       50 50 "u" "E2U+sip" "!^\\+(.*)$!sip:\\1@osu.cz!" .
1.1.1.1.1.5.8.3.7.0.2.4.e164.arpa has NAPTR record
       100 50 "u" "E2U+sip""!^\\+(.*)$!sip:\\1@cesnet.cz!" .
1.1.1.1.1.5.8.3.7.0.2.4.e164.arpa has NAPTR record
       200 50 "u" "E2U+h323" "!^\\+(.*)$!h323:\\1@gk1ext.cesnet.cz!" .
```

Here we see the contents of three NAPTR records in the ENUM DDDS application, two for the SIP service and one for the H.323 service, used for Internet telephony. The order numbers are `50` and `100` for the SIP entries and `200` for the H.323 entry,

showing how it is possible using ENUM and NAPTR records to have multiple services associated with a single telephone number, and how the provider of the NAPTR records can indicate a preferred ordering of more than one gateway providing the same service.

Note

SIP is an IETF-specified protocol used for signaling and is especially popular for facilitating the connection of multimedia clients and servers. H.323 is an ITU-specified protocol for multimedia conferencing and communication, including a signaling sub-protocol. It is widely implemented in teleconferencing equipment. In this example and those that follow, the `host` program produces output that can be used as input to a zone file for a DNS server such as BIND. As a consequence, the output shows extra escape "\" characters (which appear as "\\") that are not present in the actual DNS responses provided by the server.

To better understand how a NAPTR record's rules are applied to the AUS, we will look at the second SIP record from the preceding example. After the DNS query is performed and the NAPTR RR is received, the string appearing between the first and second `!` characters is used as a regular expression match and replacement. Thus, the string `+420738511111` is matched against the regular expression `^\+(.*)$`. According to the matching rules for regular expressions, the match is successful, so the string rewrite rule becomes `sip:\1@cesnet.cz` . The special variable `\1` is replaced with the substring matching the first regular expression contained in parenthesis characters, `()`, which in this case is everything in the AUS except for the initial `+` character. In summary, the AUS `+420738511111` is transformed into the URI `sip:420738511111@cesnet.cz`.

Once this URI is formed, the natural next step is for the driving application to contact a SIP server. However, SIP can itself be carried over different transport protocols, so the next step uses another DDDS that is tailored for SIP [RFC3263]. In this application, NAPTR records contain targets that identify the domain that should be used to perform SRV record queries. Continuing with the preceding example:

```
Linux% host -t naptr cesnet.cz
cesnet.cz has NAPTR record 200 50 "s" "SIP+D2T" "" _sip._tcp.cesnet.cz.
cesnet.cz has NAPTR record 100 50 "s" "SIP+D2U" "" _sip._udp.cesnet.cz.
```

Here we see the use of the `s` flag in the NAPTR, indicating that an SRV record is the result. The *Regexp* field is not used, so the result is a simple domain name substitution, given by the string in the *Replacement* field. The *Service* field is of the form `SIP+D2`*x* or `SIPS+D2`*x* where `SIP` and `SIPS` indicate the use of the SIP protocol and SIP protocol with security (TLS; see Chapter 18), respectively, and *x* is the single-letter identifier of the transport protocol: `U` for UDP, `T` for TCP, and `S` for SCTP [RFC4960]. In this example, the application would first attempt to look up and use the SRV record corresponding to SIP/UDP and would resort to SIP/TCP if that fails because the UDP entry has a lower preference value.

11.5.6.13 URI/URN Resolution

Although ENUM may be the most mature use of NAPTR records in the DNS, there are also DDDS applications defined for resolving URIs [RFC3404] and for persistent, location-independent URIs called *Uniform Resource Names* (URNs) [RFC2141]. All URIs (including URNs) consist of a scheme name followed by a substring compliant with semantics that are specific to the scheme. The current list of official schemes is maintained by the IANA [URI]. The URI and URN applications are so similar that it is worth considering them together. For the URI/URN DDDS application, then, the AUS is the URI or URN for which an authoritative "resolution" server is being located. The first Well-Known Rule for the URI application is simply the scheme name. For URNs, it is the name space identifier (the substring that appears after the `urn:` scheme identifier and before the next colon character). For example, `http://www.pearson.com` is a URI using the scheme (key) `http`, and the URN `urn:foo:foospace` would use `foo` as the first key. Four possible flags are currently defined: *S*, *A*, *U*, and *P*. The first three are terminal and indicate that the result is the domain name to use for fetching an SRV record, an IP address, or a URI, respectively. The *P* flag indicates that processing of the DDDS algorithm is to be discontinued and some application-specific processing (defined elsewhere) begins. All such flags are mutually exclusive. As with ENUM, the lack of any flag indicates an NTN.

Support for the URI/URN DDDS is still evolving. If we take a current (2011) look into the DNS, we can see how some of the schemes have been populated into the `uri.arpa` TLD:

```
Linux% host -t naptr http.uri.arpa
http.uri.arpa has NAPTR record 0 0 "" "" "!^http://([^:/?#]*).*$!\\1!i" .
Linux% host -t naptr ftp.uri.arpa
ftp.uri.arpa has NAPTR record 0 0 "" "" "!^ftp://([^:/?#]*).*$!\\1!i" .

Linux% host -t naptr mailto.uri.arpa
mailto.uri.arpa has NAPTR record 0 0 "" "" "!^mailto:(.*)@(.*)$!\\2!i" .
Linux% host -t naptr urn.uri.arpa
urn.uri.arpa has NAPTR record 0 0 "" "" "/urn:([^:]+)/\\1/i" .
```

The first three of these NAPTR records contain rewrite rules and no flags. Thus, they essentially indicate that the application should extract the domain name from the corresponding URI and continue the DDDS algorithm. The trailing `i` flag after the last `!` character indicates that case checking is to be performed in an insensitive way. For example, `mAiLto:person@example.com` is rewritten to be just `example.com`. The fourth record is used to extract the URN name space ID and continue processing. For URNs, there are a small number (two at present) of NAPTR records in the DNS set up in `urn.arpa`:

```
Linux% host -t naptr pin.urn.arpa
pin.urn.arpa has NAPTR record 100 100 "" "" "" pin.verisignlabs.com.
Linux% host -t naptr uci.urn.arpa
uci.urn.arpa has NAPTR record 100 100 "" "" "" uci.or.kr.
```

These URN name spaces appear to be receiving little attention at present, and it is still unclear to what extent URNs will be widely used, as there are now competing methods for expressing and locating objects using persistent identifiers (e.g., see [P10]). Nevertheless, more than 40 URN name spaces have been defined [URN], so there continues to be community interest in establishing name spaces, even though few have corresponding global, active NAPTR records.

11.5.6.14 S-NAPTR and U-NAPTR

A common issue arises when an application wishes to determine the particular host, protocol, and port number to use for reaching a service within a domain. For example, a mail-reading application running on a user's computer in the `example.com` domain may need to find a server offering the IMAP service. A convention has arisen to simply prepend the service name to the domain (e.g., `imap.example.com`). Using CNAME, A, or AAAA records is somewhat inflexible, because these record types do not convey any indication of which transport protocol or port number to use. SRV records go further by providing another layer of indirection, but their targets may contain only domain names for which an A or AAAA record is subsequently retrieved. Using NAPTR records instead provides more flexibility through an additional layer of indirection and allows for other target record types (such as SRV records) to be used.

The NAPTR structure and rewrite capabilities have caused concern for some implementers and operators given the complexity of the regular expressions. In an effort to simplify the situation yet still provide a method beyond basic SRV records for locating services, *straightforward NAPTR* (S-NAPTR) [RFC3958] specifies a DDDS application for mapping domain "labels" that contain a service name using certain simplifying restrictions on the contents of the NAPTR records.

For S-NAPTR, the AUS is a domain label for which an authoritative server for a particular service is sought. The first Well-Known Rule is the identity function. The expected output is the information necessary to contact a particular application service within a domain (e.g., protocol, host, port). Only *S* and *A* terminal flags are permitted, which indicate an SRV RR or a domain name (which is to be used to form a subsequent request for an A or AAAA RR), respectively. The service parameters are taken from a set maintained in an IANA registry [SNP], and the *Regexp* field is not used. Only the *Replacement* field is active. S-NAPTR is used in conjunction with the *Internet Registry Information Service* (IRIS) [RFC3981], an XML-based text application protocol for exchanging information pertaining to domain name and other registration information whose database is contained within the `iris.arpa` portion of the DNS name space; for example:

```
Linux% host -t naptr areg.iris.arpa
reg.iris.arpa NAPTR
       100 10 "" "AREG1:iris.xpc:iris.lwz" "" areg.nro.net.
```

This example uses S-NAPTR (no regular expression) to indicate that in order to perform an ISIS query for `AREG1`-type data (see [RFC4698]), a subsequent NAPTR query should be initiated to `areg.nro.net`.

Experience and further consideration of S-NAPTR led to the development of *URI-enabled NAPTR* (U-NAPTR) [RFC4848], which relaxes some of the restrictions of S-NAPTR but maintains all of its other features and registries. Most important, an additional *U* flag is permitted, which enables the NAPTR record target to be a URI and thus allows the use of regular expressions. This is similar to the fully generic version of NAPTR, except U-NAPTR regular expressions are restricted to the following form: `!.*!<URI>!`. That is, the entire AUS is replaced with a URI. U-NAPTR is being used in conjunction with the *Location-to-Service Translation protocol* (LoST) [RFC5222], which can be used to determine the correct service given a point of network attachment and geographical location. Such information is useful in public safety applications where geography dictates the particular jurisdiction and responsible parties that should provide emergency services.

11.5.7 Dynamic Updates (DNS UPDATE)

It is possible to dynamically update a zone, called *DNS UPDATE*, using a protocol defined in [RFC2136]. It supports the ability to specify *prerequisites* in conjunction with an update request. Prerequisites are evaluated at the server; if they are not true, the update is not performed and an error message is returned.

DNS UPDATE is accomplished by sending *dynamic update* DNS messages to an authoritative DNS server for a zone. The structure of such messages is the same as for a conventional DNS message, except the header fields and sections have different names (see Figure 11-3). The sections indicate the zone being updated, prerequisites that require various RRs to be present (or not) for the update to take effect, and the *update information*. In an update, the header mirrors the format for a query, but the *Opcode* field is set to Update (5). The header fields *ZOCOUNT*, *PRCOUNT*, *UPCOUNT*, and *ADCOUNT* contain counts of the following: zones to be updated (this will have the value 1), prerequisites to consider, updates to be made, and additional information records, respectively. [RFC2136] also defines a collection of RCODE values carried in DNS response messages capable of indicating conditions relating to problems with the prerequisites or server (values 6–10 in Table 11-2).

The zone section of an update message (see Figure 11-7) indicates the zone's name, a type, and a class. The type value will be 6 to indicate the presence of an SOA record, which identifies the zone. The class value will be 1 (Internet) for any update message with which we are concerned. All records being updated must be in the same zone.

The prerequisite section of an update message contains one or more prerequisites, expressed using the format for RRs we discussed previously in Section 11.5.5. There are five types of prerequisites: *RRSet exists* (value-dependent and value-independent varieties), *RRSet does not exist*, *name is in use*, and *name is not in use*. Recall that an RRSet is a group of RRs from the same zone sharing a common name, class, and type. To express the semantics of a prerequisite, a combination of an RR's class, type, and RDATA values are set according to Table 11-5.

The *RRSet exists* type means that at least one RRSet exists in the zone specified in the zone section that matches the name and type of the corresponding RR in

Table 11-5 *RR Class* and *Type* fields used in prerequisite section to indicate prerequisite type

Prerequisite Type (Semantics)	Class Setting	Type Setting	RDATA Setting
RRSet exists (value-independent)	ANY	Same as zone's type	Empty
RRSet exists (value-dependent)	Same as zone's class	Type being checked	RRSet being checked
RRSet does not exist	NONE	Type being checked	Empty
Name is in use	ANY	ANY	Empty
Name is not in use	NONE	ANY	Empty

the prerequisite section. In the value-dependent case, the prerequisite is true only if the matching RRs also contain matching RDATA values. The *RRSet does not exist* type means that no RRSet in the zone specified in the zone section matches the name and type of the RR in the Prerequisites section. The last two cases (*Name is in use* and *Name is not in use*) refer only to the domain name; the type value is not used. The values for NONE and ANY as DNS classes are 254 and 255, respectively.

Following the Prerequisite section, the Update section contains RRs to be added or deleted from the zone specified in the zone section. There are four types of updates, encoded as an RR with various combinations of values in the *Class*, *Type*, and *RDATA* fields, as indicated in Table 11-6.

Table 11-6 *RR Class* and *Type* fields used in Update section to indicate update type

Use	Class Setting	Type Setting	RDATA
Add RR to RRSet	Same as zone's class	Type of RR being added	RDATA of RR being added
Delete RRSet	ANY	Type of RRSet to delete	Empty (TTL and RDLENGTH also zero)
Delete all RRSets from a name	ANY	ANY	Empty (TTL and RDLENGTH also zero)
Delete RR from RRSet	NONE	Type of RR being deleted	Matching RDATA to delete

The update section contains a collection of RRs that are processed provided no errors have occurred due to prerequisites or server problems. Each RR encodes an addition or deletion operation. Modifications can be performed as a deletion followed by an addition. To see an example of DNS UPDATE, we can induce a Windows machine to perform a dynamic DNS update using the following command:

```
C:\> ipconfig /registerdns
```

Windows clients issue updates for their computer name and domain name by default, but this behavior can also be enabled for IPv4 on a per-DNS-suffix basis by checking the box labeled "Use this connection's DNS suffix in DNS registration"

under the DNS section of the Advanced TCP/IP Settings, found on the General tab of the Internet Protocol (TCP/IP) Properties menu associated with each interface enabled for TCP/IP. For IPv6, the same procedure is used, but on the IPv6 Properties menu. In the example shown in Figure 11-16, we can see how the machine named `vista` updates the local zone `dyn.home` as it issues the DNS update message shown.

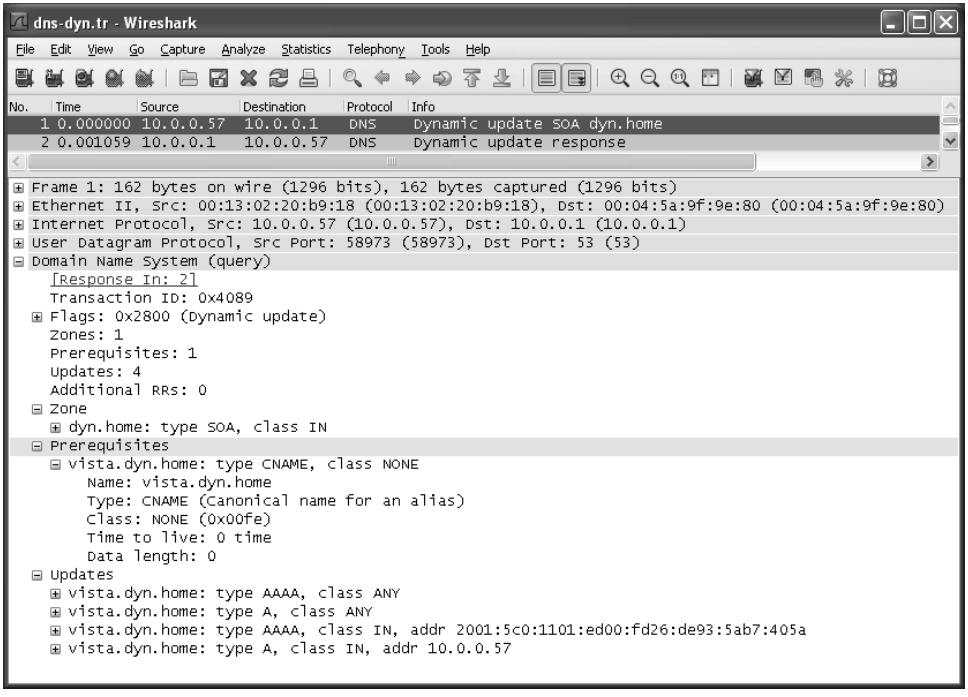

Figure 11-16 A DNS dynamic update contains an SOA record in the zone section and RRs in the update section. This case includes new IPv4 and IPv6 addresses for the host `vista.dyn.home`.

Figure 11-16 shows how a dynamic update is encoded. The DNS server at 10.0.0.1 (running BIND9 [AL06] in this example) is configured to allow dynamic updates. The zone section contains an SOA record identifying the zone to be updated (`vista.dyn.home`). The prerequisite section contains an RR with a zero-length RDATA section and 0 TTL value. The RR corresponds to the type of prerequisite in the third row of Table 11-5 (*RRset does not exist*) because its type is not ANY (it is CNAME) and its class is set to NONE (254).

In this particular case, the addresses 10.0.0.57 and 2001:5c0:1101:ed00:fd26:de93:5ab7:405a are to be associated with the name `vista.dyn.home`. This is accomplished by first deleting the AAAA and A RRSets (corresponding to row 2 in Table 11-6), and then adding the AAAA and A RRSets (corresponding to row 1 in Table 11-6) for the desired addresses.

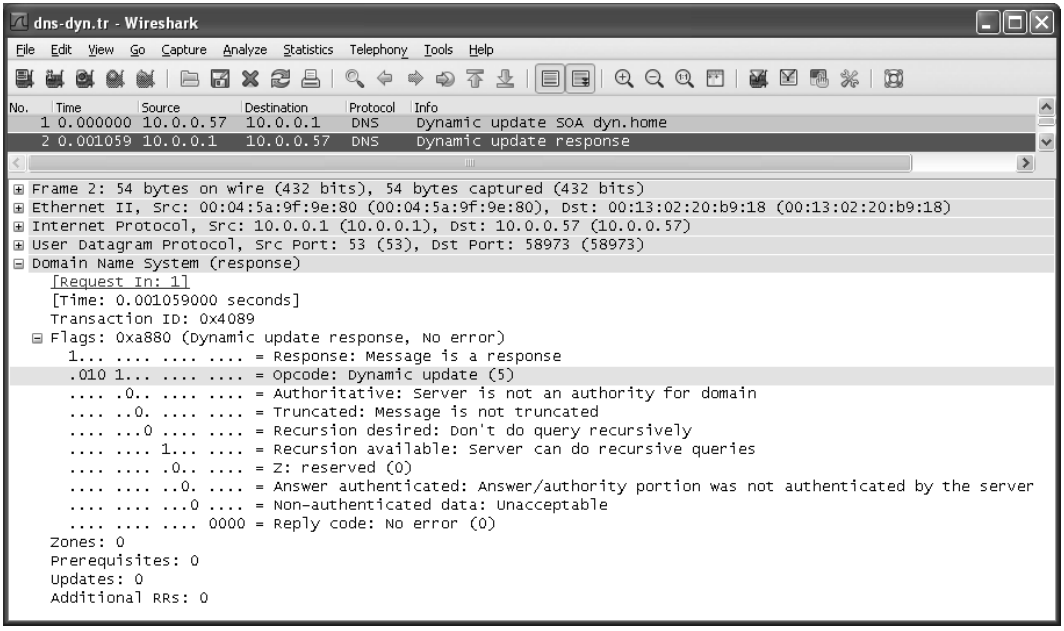

Figure 11-17 The response to a dynamic update request includes a transaction ID and status flag set.

Responses to DNS updates are straightforward and compact. The response for the update shown in Figure 11-16 is illustrated in Figure 11-17.

The *Flags* field indicates a successful update (no error). The transaction ID (0x4089) is used to ensure that the update response matches a corresponding request. Note that on Linux, the `nsupdate` program can be used to update a cooperative DNS server. DNS servers cooperate with a requested update only if an authentication and access control procedure indicates that the request is acceptable. This can be as simple as nothing or listing the IP addresses of clients at the server, neither of which is very secure, or using somewhat more complex and secure methods that provide *transaction authentication* (see TSIG and SIG(0) in Chapter 18).

11.5.8 Zone Transfers and DNS NOTIFY

A zone transfer is used to copy a set of RRs for a zone from one server to another (generally from the master server to slave servers). The purpose of doing so is to keep multiple servers in sync with respect to a zone's contents. Multiple servers provide resiliency to failure, in case a server should go down. Performance can also be improved as multiple servers can be used to share the processing load for incoming queries. Finally, the latency of a DNS query/response can potentially be reduced if servers are placed in locations close to clients (i.e., where the network latency between resolver and server is small).

As originally specified, zone transfers are initiated after *polling*, where slaves periodically contact masters to see if a zone transfer is necessary by comparing the zones' version numbers. A later method says if a zone transfer needs to be initiated using an asynchronous update mechanism when the zone contents change. This is called *DNS NOTIFY*. Once a zone transfer is initiated, either the entire zone is transferred (using DNS AXFR messages) [RFC5936], or an *incremental zone transfer* option may be used (using DNS IXFR messages) [RFC1995]. The general scheme operates according to the illustration in Figure 11-18.

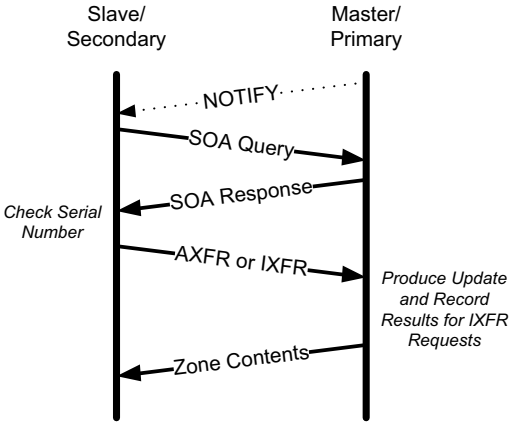

Figure 11-18 A DNS zone transfer copies the contents of zones between servers. An optional notification can cause a slave to request a full or incremental zone transfer.

We will now have a closer look at each of the options, including full and incremental zone transfers, plus DNS Notify.

11.5.8.1 Full Zone Transfers (AXFR Messages)

Full zone transfers are controlled by the zone transfer parameters carried in a zone's SOA record: primary name server, serial number, and the refresh, retry, and expire intervals. When configured, a slave server attempts to contact the primary server to see if a zone transfer is necessary. Contacts are attempted periodically, according to the refresh interval. They are also attempted when a server first starts. If a contact is not successful (no response from the server), retries are attempted periodically according to the retry interval (generally shorter than the refresh interval). The entire zone contents are flushed if not refreshed within the expire interval, effectively incapacitating the server for the zone.

An All Zone Transfer (AXFR) DNS message (a standard query containing type AXFR in the Question section) is used to request a complete zone transfer using TCP. To see such a message, we may arrange for a request to be initiated using the `host` program in our local network:

```
Linux% host -l home.
Using domain server:
Name: 10.0.0.1
Address: 10.0.0.1#53
Aliases:

home name server gw.home.
ap.home has address 10.0.0.6
gw.home has address 10.0.0.1
...
```

The `-l` flag asks the `host` program to perform a full zone transfer from a local DNS server. The program initiates a TCP-based query/response dialogue, illustrated in Figure 11-19.

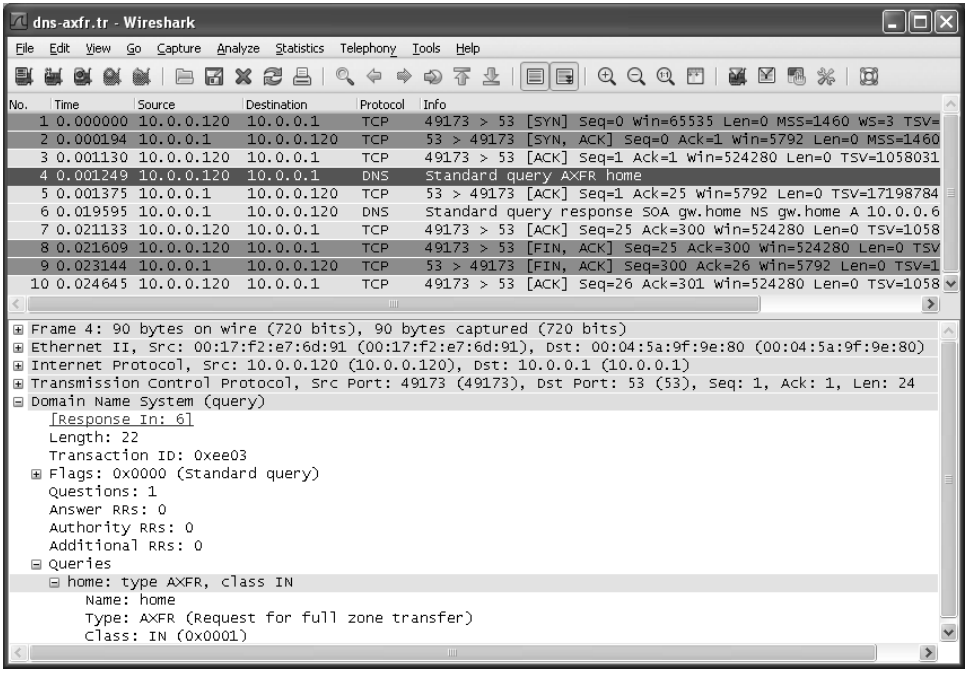

Figure 11-19 A DNS request for a full zone transfer uses the AXFR record type and TCP as a transport protocol.

In Figure 11-19 we can see how the zone transfer takes place using TCP. The first three TCP segments are part of the standard TCP connection establishment process (see Chapter 13). The fourth (decoded) packet is the request. It is a normal DNS standard, with type AXFR and class IN (Internet). The query is for the domain name `home`. The response to this query is contained in message 6, following the TCP ACK (see Figure 11-20).

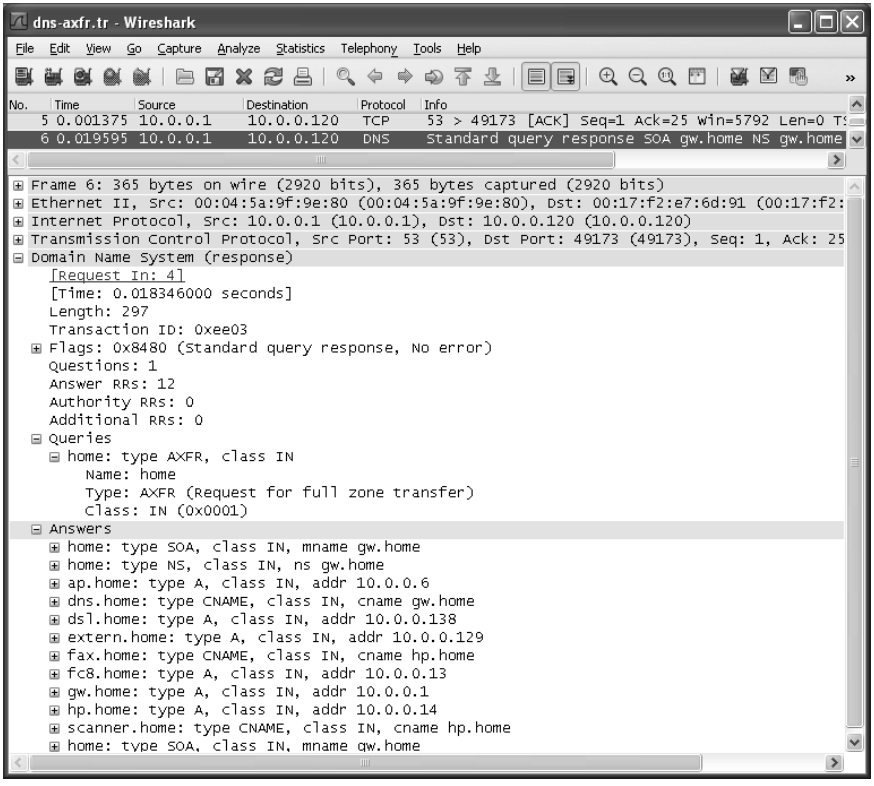

Figure 11-20 The successful response for a full zone transfer request includes all of the records for the zone. The transaction takes place using TCP, as the zone contents may be large and a reliable copy is required.

In Figure 11-20 we can see how the entire zone is carried in the response. After receiving the response, the client's TCP ACKs the data and initiates a TCP connection close. The connection is closed gracefully using the FIN-ACK handshake (packets 8–10). See Chapter 13 for more details on the standard TCP connection establishment and clearing.

Although it used to be possible to perform such zone transfers with virtually any DNS server, they are now typically restricted to the authoritative servers in a zone (e.g., those listed in NS records for the zone). The reason for this restriction is privacy and security—knowledge of the hosts within the zone might help an attacker target particular services or hosts.

11.5.8.2 Incremental Zone Transfers (IXFR Messages)

To improve the efficiency of zone transfers, [RFC1995] defines the use of *incremental* zone transfers. Using incremental zone transfers and the IXFR message type, only the changes in a zone are provided. To execute an incremental zone transfer, the client (e.g., slave server) must provide its current serial number for the zone. In

the following example, we can emulate a requesting server by providing the serial number and using the `dig` program:

```
Linux% dig +short @10.0.0.1 -t ixfr=1997022700 home.
gw.home. hostmaster.gw.home. 1997022700 10800 15 604800 10800
```

The command line indicates that output from the command should be short, `10.0.0.1` is the address of the DNS server to use, and an incremental zone transfer starting with serial number `1997022700` should be performed. This example creates an exchange similar to the one illustrated in Figures 11-19 and 11-20 for AXFR, except in this case the serial number of the request matches the current serial number (see Figure 11-21).

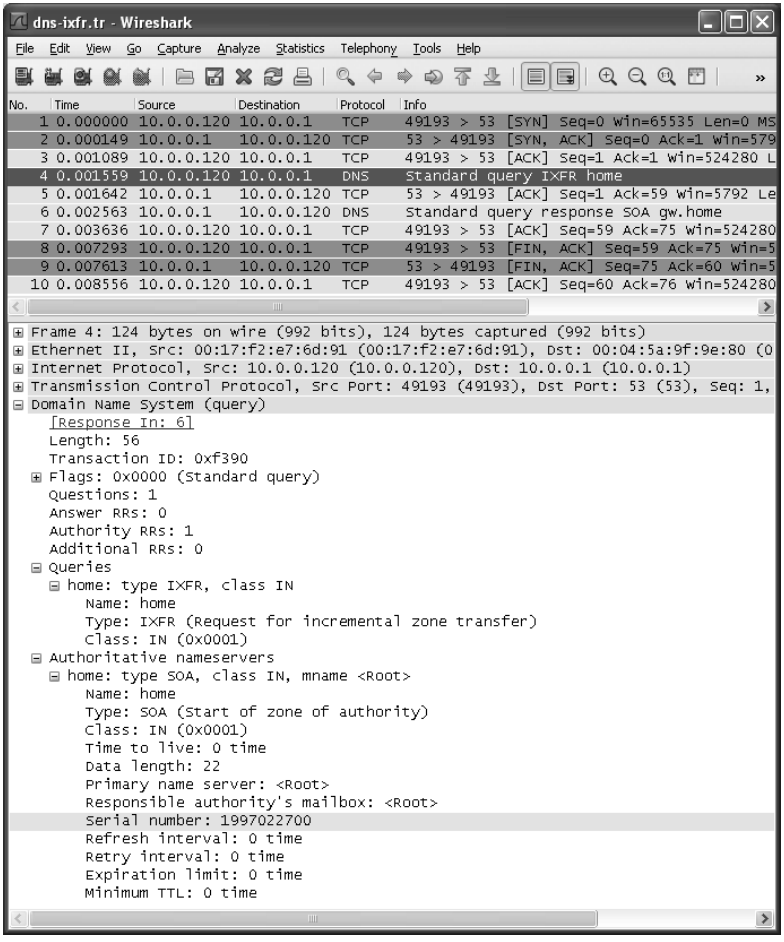

Figure 11-21 An incremental zone transfer request (IXFR record type) carried on TCP. The serial number is used to determine which records, if any, have changed since an earlier zone transfer took place.

Figure 11-22 shows how the IXFR request includes a mostly empty SOA RR in the authority section. The SOA record includes the serial number specified (`1997022700`). The response (packet 6) contains no real information because this serial number matches the current one at the server.

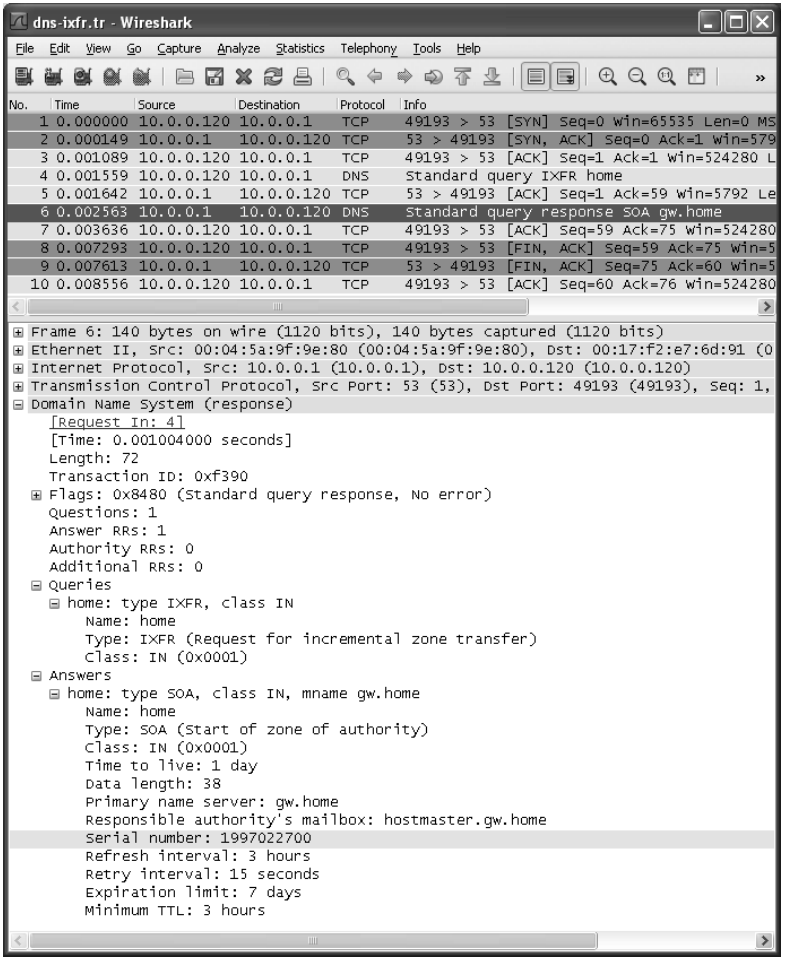

Figure 11-22 The response to an IXFR request when the serial number is current contains only an SOA record and no additional information.

The response in Figure 11-22 contains only the SOA RR in the answer section. Unlike the one contained in the query, this one is filled in with the complete SOA fields (e.g., mailbox, zone transfer parameters). However, there are no additional answers because the current serial number for the zone matches that of the request. Thus, the requesting client is assumed to be up-to-date and not in need of any additional information or a zone transfer.

11.5.8.3 DNS NOTIFY

As mentioned previously, polling has traditionally been used to determine the need for zone transfers, meaning that the slave servers would check with a master periodically (the "refresh" interval) to see if the zone had been updated (indicated by a different serial number), in which case a zone transfer would be initiated. This is a somewhat wasteful process because many useless polls may occur before the zone is updated. To improve the situation, [RFC1996] developed the *DNS NOTIFY* mechanism. DNS NOTIFY allows a server with modified zone contents to notify slave servers that an update has been made and a zone transfer should be initiated. More specifically, if enabled, a notification message is sent to a set of interested servers if the SOA RR for a zone changes (e.g., if the serial number increases). This allows zone transfers to be initiated easily when required. Using a local (home) name server, we can see how this works (see Figure 11-23).

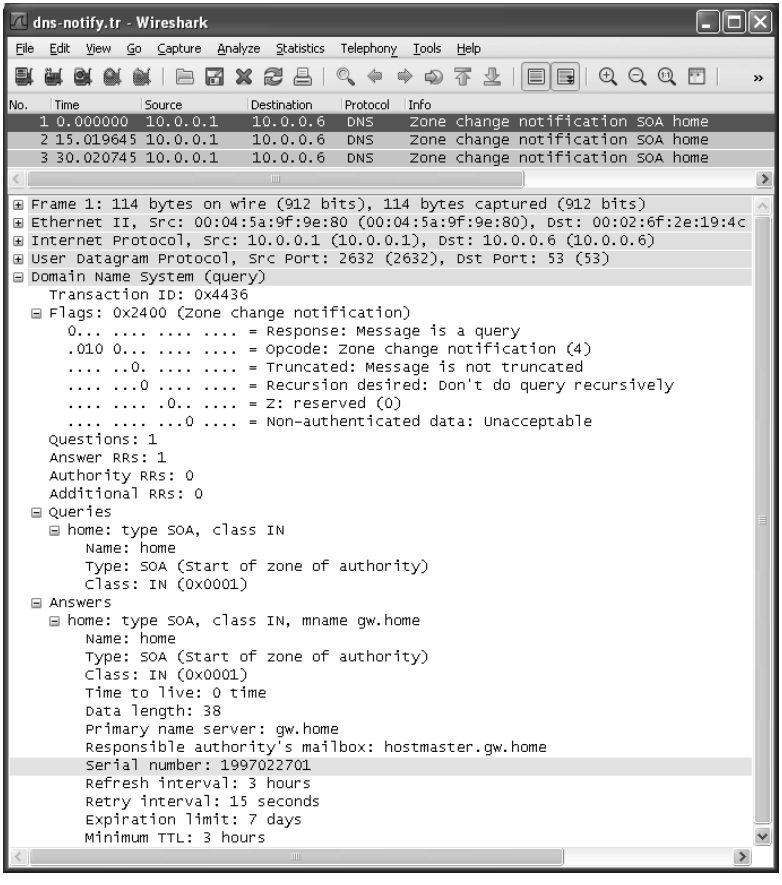

Figure 11-23 A DNS NOTIFY indicating an update to the zone file. There are two retransmissions spaced 15s apart (contrary to the method suggested in the standard).

This example illustrates the simple DNS NOTIFY message sent to a host in the server's *notify set* of servers that should be informed of a zone change. The message is a UDP/IPv4 DNS query message with the *Flags* field indicating a zone change notification. The query section contains the type and class for an SOA record, and the answer section contains the current SOA RR for the zone (with TTL 0), including the serial number. This provides sufficient information for a notified server to determine that a zone transfer may be necessary. Note that a single server may receive notifications from multiple other servers as they update their zone information. This does not present a problem for the protocol's operation.

The DNS NOTIFY mechanism defaults to using UDP, an unreliable protocol. In this particular example, the notify set contains only the address 10.0.0.11, which does not run a DNS server. Consequently, the message is resent every 15s hoping for a response that never arrives.

Note

The time between retransmissions and the total number of retransmissions to attempt are suggested by [RFC1996] to be 60s and five retransmissions, respectively. It also suggests that a timer backoff method (additive or exponential) be used. Here we can see that the BIND9 implementation fails to respect these suggestions, as the two retransmissions are 15s apart.

Responses are simply DNS response messages with no useful information except the transaction ID; they are used only to complete the protocol and cancel retransmissions at the sending server.

11.6 Sort Lists, Round-Robin, and Split DNS

So far we have discussed how domain names are set up, the types of resource records DNS supports, and the DNS protocol used to fetch and update a zone. One subtle point to consider is what data is returned and in what order in response to a DNS query. A DNS server could return all matching data to any client in whatever order the server finds most convenient. However, special configuration options and behaviors are available in most DNS server software to achieve certain operational, privacy, or performance goals. Consider the topology shown in Figure 11-24.

The type of topology shown in Figure 11-24 is typical of a small enterprise. There is a private network and a public network including a DNS server. In addition, there is a pair of hosts on the DMZ (A and B), one on the internal network (C) and one on the Internet (R). A multihomed host (M) spans the DMZ and internal networks. M therefore has two IP addresses drawn from two different network prefixes.

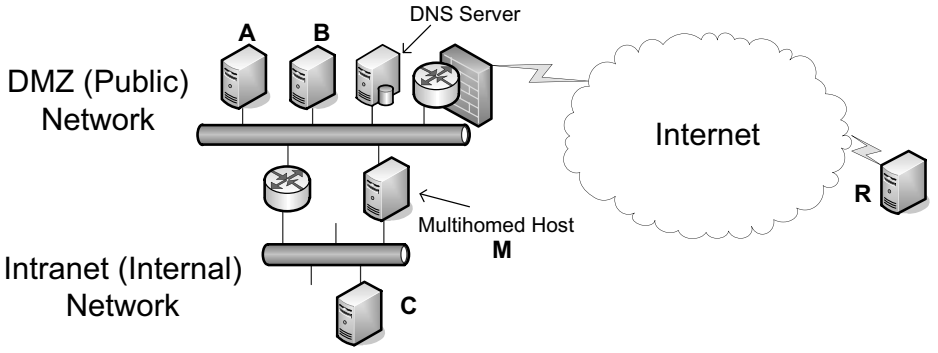

Figure 11-24 In a small enterprise topology, DNS may be configured to return different addresses depending on the requesting IP address.

A host wishing to contact M performs a DNS lookup that returns two addresses—one associated with the internal network and one with the DMZ. Naturally, it would be more efficient if A, B, and R reached M via the DMZ and C reached M via the internal network. This generally happens if the DNS server orders its returned address records based on the source IP address of the request. (It could also use the destination IP address, especially if M uses multiple IP addresses from different subnets on the same network interface.) If the requesting system uses a source IP address with the same network prefix as the source of a returning address record, the DNS server places the set of such matching records early in the returned message. This behavior encourages the client to find the "closest" IP address for a particular server it is attempting to contact, because most simple applications attempt to contact the first address found among the returned address records. The precise behavior can usually be controlled using a so-called `sortlist` or `rrset-order` directive (options used in configuration files for resolvers and servers). Such sorting behavior may also happen automatically if performed by the DNS server software by default.

A somewhat related situation arises when one service is offered using more than one server such that the incoming connections are load-balanced (i.e., divided among the servers). In the preceding example, imagine that a service is offered on both A and B. Such a service may be identified by the URL `http://www.example.com`. Requesting clients (like R) perform a DNS query on the domain name `www.example.com`, and the DNS server eventually returns a set of address records. To achieve load balancing, the DNS server may be configured to use *DNS round-robin*, which means that the server permutes the order of the returned address records. Doing so encourages each new client to access the service on a different server from the previous client. While this helps to balance load, it is far from perfect. When records are cached, the desired effect may not occur because of reuse of existing cached address records. In addition, this scheme may balance the number of connections well across servers, but not the load. Different

connections can have radically different processing requirements, so the true processing load is likely to remain unbalanced unless the particular service always has the same processing requirements.

A final consideration regarding the data returned by a DNS server is support for privacy. In this example, we may wish to arrange for hosts within the enterprise to be able to retrieve resource records for every computer in the network, while we limit the set of systems that remain visible to R. A technique for implementing this goal is called *split DNS*. In split DNS, the set of resource records returned in response to a query is dependent on the identity of the client and possibly query destination address. Most often, the client is identified by IP address or address prefix. With split DNS, we could arrange for any host in the enterprise (i.e., those sharing a set of prefixes) to be provided with the entire DNS database, whereas those outside are given visibility only to A and B, where the main Web service is offered.

11.7 Open DNS Servers and DynDNS

Many home users are assigned a single IPv4 address by their ISP, and this address may change over time as the user's computer or home gateway connects, disconnects, and reconnects to the Internet. Consequently, it is often difficult for the user to establish a DNS entry that allows for running services that are visible from the Internet. A number of so-called open *Dynamic DNS* (DDNS) servers are available that support a special update protocol called the *DNS Update API* [DYNDNS], whereby a user may update an entry in a provider's DNS server given a preregistration or account. This scheme does *not* use the [RFC2136] DNS UPDATE protocol described earlier but is instead a separate application-layer protocol.

To use the service, a DDNS client program (e.g., `inadyn` or `ddclient` on Linux and `DynDNS Updater` for Windows) runs on the client system, which could also be a user's home router. Most often, these programs are configured with login information used to access a remote DDNS service. When the service is invoked, the client program contacts the server, provides the current global IP address of its host (the one assigned by an ISP, often a NAT mapped address), and goes quiescent. After that, it periodically renews the information with the server. Doing so allows the server to clear the information if an update is not received within a certain time interval. Such services include those provided at the following Web sites (as of 2011): `http://www.dyndns.com/services/dns/dyndns`, `http://freedns.afraid.org`, and `http://www.no-ip.com/services/managed_dns/free_dynamic_dns.html`.

11.8 Transparency and Extensibility

The DNS is one of the most ubiquitous services on the Internet and has been an attractive service to consider as a basis for adding new capabilities through

extensions. There are, for example, numerous record types such as TXT, SRV, and even A (e.g., see [RFC5782]) that could be used for encoding data useful for various future services. [RFC5507] considers various methods for extending the DNS, ultimately concluding that creation and implementation of new RR types is the most attractive approach. Thanks to an earlier specification [RFC3597], there is a standard method for handling unknown RR types as opaque data. That is, they are not interpreted unless recognized; the processing is *transparent*. This allows for new RR types to be carried along without causing negative impact on the processing of existing RR types.

One complication with preserving transparency is the encoding of embedded domain names and compression. For known RR types, embedded domain names are permitted to have their cases altered in order to achieve compression with compression labels. Owner domain names (the "keys" of queries) are always subject to compression. For unknown RR types, however, embedded domain names are not permitted to use compression labels. In addition, future RR types that contain embedded domain names are likewise prohibited (see Section 4 of [RFC3597]). Unknown types can still be compared (e.g., for dynamic updates) in a bitwise fashion. This implies that any embedded domain names are compared in a case-*sensitive* manner [RFC4343], contrary to most other DNS operations. This same situation appears for embedded domain names used with TXT records.

A different issue arises regarding transparency when new forms of servers and proxies are introduced that process DNS traffic. It is now relatively common practice to include a *DNS proxy* colocated inside a home gateway or firewall. A typical proxy handles incoming DNS requests from a user's home network and forwards the request to an ISP-provided name server. It also receives returned information and may or may not cache the results. Historically, some proxies have tried to do more than merely relay requests and replies, and this has caused some problems with DNS interoperability. [RFC5625] specifies the proper operation of a DNS proxy, essentially requiring DNS RRs to be uninterpreted and merely relayed by the proxy. In cases where packet truncation cannot be avoided, any such proxy must set the *TC* bit field to indicate that some DNS data was removed. Furthermore, any such proxies should be prepared to handle TCP requests, as this is the conventional fallback mechanism when a previous UDP-based request was truncated and is required by [RFC5966].

11.9 Translating DNS from IPv4 to IPv6 (DNS64)

In Chapter 7 we described a framework for translating IP datagrams back and forth between IPv4 and IPv6. Translators supporting such capabilities are envisioned to be deployed with a related capability that translates between DNS A and AAAA records [RFC6147], allowing IPv6-only clients to access DNS information that appears in A records (e.g., in the IPv4 Internet). The capability is called DNS64, and one of its proposed deployment scenarios (called "DNS64 in DNS recursive-resolver mode") is illustrated in Figure 11-25.

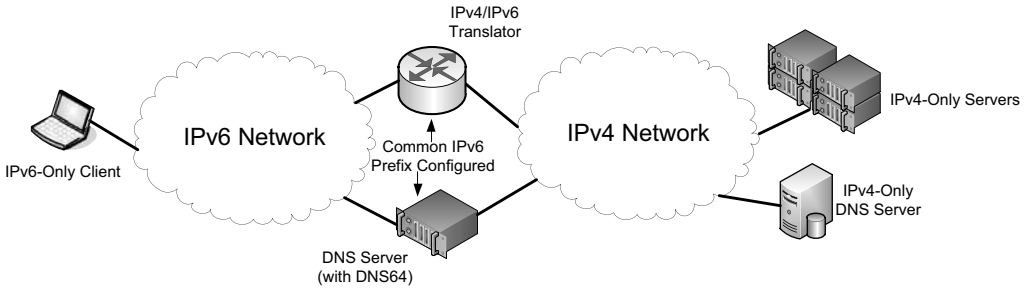

Figure 11-25 DNS64 translates A records to AAAA records and works together with an IPv4/IPv6 translator to allow IPv6-only clients to access services in IPv4 networks.

As shown in Figure 11-25, DNS64 is used in conjunction with an IPv4/IPv6 translator (see Chapter 7). Each device is configured with one or more common IPv6 prefixes used in creating IPv4-embedded addresses. Each prefix may be a Network-Specific Prefix (e.g., that is owned by an operator) or the Well-Known Prefix (64:ff9b::/96). The DNS64 device acts as a caching DNS server. IPv6-only clients use it as the primary DNS server and are able to request AAAA records for domain names. DNS64 converts such requests to requests for both A and AAAA records on its IPv4 side. If no AAAA records are returned, DNS64 provides *synthetic* AAAA records by forming an IPv4-embedded address based on the configured prefix and the contents of each A record it retrieves. DNS64 also responds to PTR queries for any of the IPv6 prefixes it uses for synthesizing AAAA RRs.

To implement AAA RR synthesis in a DNS64 device, only the answer section of a DNS message is effectively altered. Other sections remain as they appear when retrieved on the IPv4 side. In cases of CNAME or DNAME chains, the chain is followed recursively until an A or AAAA record is found and the elements of the chain are included in the response. In addition, DNS64 may be configured so as to avoid synthesis for particular excluded IPv6 or IPv4 address ranges. This prevents certain anomalous behavior (e.g., forming IPv4-embedded addresses based on special-use IPv4 addresses). Note that DNS64 has subtle interactions with DNSSEC; these issues are covered in Chapter 18.

11.10 LLMNR and mDNS

The ordinary DNS system requires a set of DNS servers to be configured to provide mappings between names and addresses, and possibly other information. Sometimes this is too much overhead when only a few local hosts wish to communicate. In cases where a DNS server is not available (e.g., a quickly formed ad hoc network of clients that connect only to each other), a special local version of DNS called *Link-Local Multicast Name Resolution* (LLMRR) [RFC4795] may be available. It is a (nonstandard) protocol based on DNS developed by Microsoft and used in local

environments to help discover devices on a local area network, such as printers and file servers. It is supported in Windows Vista, Server 2008, and 7. It uses UDP port 5355 with the IPv4 multicast address 224.0.0.252 and IPv6 address ff02::1:3. The servers also use TCP on port 5355 from whatever unicast IP address they respond from.

Multicast DNS (mDNS) [IDMDNS] is another form of local DNS-like capability developed by Apple. When it is combined with the DNS Service Discovery protocol, Apple calls the resulting framework Bonjour. mDNS uses DNS messages carried over local multicast addresses. It uses UDP with port 5353. It specifies that the special TLD `.local` is to be treated with special semantics. The `.local` TLD is link-local in scope. Any DNS queries for domain names in this TLD are sent to the mDNS IPv4 address 224.0.0.251 or the IPv6 address ff02::fb. Queries for other domains may optionally be sent to these multicast addresses. Allowing link-local servers to respond to mappings for global names can raise significant security concerns. To combat this problem, DNSSEC can be employed (see Chapter 18). mDNS supports autonomous assignment of names in the `.local` pseudo-TLD, although this pseudo-TLD has not been officially reserved for this purpose [RFC2606]. Thus, hosts on small networks such as home LANs can be assigned convenient names such as `printer.local`, `fileserver.local`, `camera1.local`, `kevinlaptop.local`, and the like. A mechanism in mDNS is used to detect and resolve conflicts.

11.11 LDAP

So far we have discussed DNS and local name services that resemble DNS. To support richer queries and data manipulations, there is a more general directory service we mentioned earlier called LDAP [RFC4510]. LDAP (now LDAPv3) is an application protocol for the Internet that provides access to general directories (e.g., "white pages") in accordance with the X.500 (1993) [X500] data and service models. It provides the ability to search, modify, add, compare, and remove entries based on user-selected patterns. An LDAP directory is a tree of directory entries, where each entry consists of a set of attributes. As TCP/IP has become more popular, LDAP has evolved from its roots to work in conjunction with DNS. For example, a query about directory entries matching the chancellor's office at MIT could be formed using the LDAP search tool `ldapsearch` (Microsoft has a comparable tool called `ldp` available as a support tool from its Web site), which works as follows:

```
Linux% ldapsearch -x -h ldap.mit.edu -b "dc=mit,dc=edu" \
"(ou=*Chancellor*)"
# extended LDIF
#
# LDAPv3
# base <dc=mit,dc=edu> with scope sub
```

```
# filter: (ou=*Chancellor*)
# requesting: ALL
#
.....
```

The command line indicates that the server `ldap.mit.edu` should be contacted without using any special authentication protocol (`-x` option). While a complete discussion of LDAP is well beyond the scope of this chapter (and book!), the partial output shows how the `dc` (domain component) attribute is used to link LDAP data with the DNS. Each `dc` component holds one DNS label, and together they can be used to encode an entire domain name, which is used as the "base" portion for the LDAP query. Using this convention, it is not especially difficult to form valid LDAP queries. In this case, it is for the organizational unit (`ou`) containing the word `Chancellor`. Note that wildcards can be used.

LDAP servers are used most often within enterprises to hold directory information such as location, telephone number, and organizational unit. Microsoft's Active Directory product includes LDAP capabilities and is used extensively for managing user accounts, services, and access rights in large enterprises using Windows. Some LDAP servers (such as MIT's and those of many other universities) are also available through the public Internet.

11.12 Attacks on the DNS

The DNS is a critical component of the Internet and has been the object of several attacks and countermeasures over the years [RFC3833]. Relatively recently, a global effort called DNS Security (DNSSEC) has made substantial progress in adding strong authentication to DNS operations. We defer the detailed discussion of how DNSSEC works to Chapter 18, where we also cover the necessary cryptography background. We now explore some of the attacks that have been waged against the DNS.

There have been two main forms of attacks against the DNS. The first form involves a DoS attack where the DNS is rendered inoperative because of overloading of important DNS servers, such as the root or TLD servers. The second form alters the contents of resource records or masquerades as an official DNS server but responds with bogus resource records, thereby causing hosts to contact the incorrect IP address when attempting to connect to another machine (e.g., a Web site such as a bank).

The first major DoS attack on DNS took place in early 2001. The attack involved generating many requests for the MX records of `AOL.COM`. The attacker generated DNS requests for an MX record using forged source IP addresses. The request is a relatively small packet, whereas the response is larger (by about a factor of 20), so this type of attack is called an *amplification* attack because the amount of bandwidth consumed as the result of the attack is greater than the amount used in generating the attack by a significant factor. The responses are directed at the

IP address contained in the request packets, so the attacker could essentially cause the response traffic to be directed wherever (s)he intended. The attack is documented in detail in a CERT incident note [CIN].

A form of attack involving modification of the data within DNS was reported in late 2008 [CKB] and is now known as the *Kaminsky Attack*. It involves *cache poisoning*, where the cached contents of a DNS server are replaced with erroneous or forged data and ultimately served to the resolvers on end hosts. In one variant, an attacker responds to a caching server's query for an A record with an NS record for the domain using a particular host domain name. The host's IP address (chosen by the attacker) is also provided in the additional information section of the DNS response. The host domain name may or may not share the same subdomains as the original DNS request. The main risk associated with this form of attack is that clients that depend on proper DNS name-to-address resolution may be directed to fake servers. If such servers are intentionally configured to mimic the original host (e.g., masquerading as a bank's Web server), users may unwittingly trust the masquerading server and divulge sensitive information. Mitigation techniques for this and other related attacks are given by [RFC5452]. One approach not described in [RFC5452] called *DNS-0x20* [D08] involves encoding a nonce in the 0x20 bit position of each character in the Query Name part of a question section that is echoed back in the corresponding area of each response. This is made possible because, although domain names are compared in a case-insensitive way, servers tend to return an exact copy of the Query Name when forming responses. If the case of the owner's name is intentionally mixed up in the query, an unsolicited response will have difficulty reproducing the nonce, and can more readily be identified (and ignored).

11.13 Summary

The DNS is an essential part of the Internet, and DNS technology is widely used in private networks as well. The DNS name space is worldwide in scope and is divided into a hierarchy starting with top-level domains (TLDs). Domain names can be represented in multiple languages and scripts using internationalized domain names (IDNs). Applications use resolvers to contact one or more DNS servers to perform lookup tasks against a zone database, such as converting a host name to an IP address and vice versa. Resolvers then contact a local name server, and this server may act recursively to contact one of the root servers or other servers to fulfill the request. Most DNS servers, and some resolvers, cache information learned in order to provide it to subsequent clients for some period of time called the time to live (TTL). Queries and responses use a special DNS protocol that works with either TCP or UDP. The protocol also works with either IPv4 or IPv6, or any mixture of the two.

All DNS queries and responses have the same basic message format that includes questions, answers, authority information, and additional information.

Resource records are used to hold most DNS information, and there are many such types: addresses, mail exchange points, pointers to names, among others. In the Internet, most DNS messages are carried using UDP/IPv4 and are limited to 512 bytes in length, but a special extension option (EDNS0) provides for longer messages and is required to support DNS security (DNSSEC), which we discuss in detail in Chapter 18.

DNS supports some special features such as zone transfers and dynamic updates. Zone transfers (complete or incremental) are used to allow redundant slave servers to synchronize the zone contents with a master server, primarily for redundancy. Dynamic updates allow zone contents to be modified by an application using an online protocol. There are really two forms of this capability, one standardized by [RFC2136] and used in enterprises and a nonstandard but very popular dynamic DNS capability that allows users assigned temporary IP addresses (e.g., on cable or DSL) to obtain a DNS entry so that services they provide can be found by name throughout the world.

DNS has been the subject of numerous attacks, ranging from DoS attacks that leave the DNS with limited capability, to cache poisoning attacks that can be used to make malicious servers appear to be legitimate. Various techniques have arisen to combat this problem, including cryptographic techniques (covered in Chapter 18) and modifications to DNS servers to be less accepting of unsolicited DNS responses.

11.14 References

[AL06] P. Albitz and C. Liu, *DNS and BIND, Fifth Edition* (O'Reilly Media, Inc., 2006).

[CIN] http://www.cert.org/incident_notes/IN-2000-04.html

[CKB] http://www.kb.cert.org/vuls/id/800113

[D08] D. Dagon et al., "Increased DNS Forgery Resistance Through 0x20-bit Encoding," *Proc. ACM CCS*, Oct. 2008.

[DNSPARAM] http://www.iana.org/assignments/dns-parameters

[DR97] D. Dougherty and A. Robbins, *sed & awk, Second Edition* (O'Reilly Media, 1997).

[DYNDNS] http://www.dyndns.com/about/technology

[ENUM] http://www.iana.org/assignments/enum-services

[GTLD] http://www.iana.org/domains/root/db

[ICANN] http://www.icann.org/en/tlds

[IDDN] S. Rose and W. Wijngaards, "Update to DNAME Redirection in the DNS," Internet draft-ietf-dnsext-rfc2672bis-dname, work in progress, July 2011.

[IDMDNS] S. Cheshire and M. Krochmal, "Multicast DNS," Internet draft-cheshire-dnsext-multicastdns, work in progress, Feb. 2011.

[IIDN] http://www.icann.org/en/topics/idn

[ISO3166] International Organization for Standardization, "International Standard for Country Codes," ISO 3166-1, 2006.

[ISPR] http://www.iana.org/assignments/service-names-port-numbers

[J02] J. Jung et al., "DNS Performance and the Effectiveness of Caching," *IEEE/ACM Transactions on Networking*, 10(5), Oct. 2002.

[MD88] P. Mockapetris and K. Dunlap, "Development of the Domain Name System," *Proc. ACM SIGCOMM*, Aug. 1988.

[P10] N. Paskin, "Digital Object Identifier (DOI©) System," *Encyclopedia of Library and Information Sciences, Third Edition* (Taylor and Francis, 2010).

[R06] H. Rice, "ENUM—The Mapping of Telephone Numbers to the Internet," *The Telecommunications Review*, 17, Aug. 2006.

[RFC1035] P. Mockapetris, "Domain Names—Implementation and Specification," Internet RFC 1035/STD 0013, Nov. 1987.

[RFC1464] R. Rosenbaum, "Using the Domain Name System to Store Arbitrary String Attributes," Internet RFC 1464 (experimental), May 1993.

[RFC1536] A. Kumar et al., "Common DNS Implementation Errors and Suggested Fixes," Internet RFC 1536 (informational), Oct. 1993.

[RFC1912] D. Barr, "Common DNS Operational and Configuration Errors," Internet RFC 1912 (informational), Feb. 1996.

[RFC1918] Y. Rekhter, B. Moskowitz, D. Karrenberg, G. J. de Groot, and E. Lear, "Address Allocation for Private Internets," RFC 1918/BCP 0005, Feb. 1996.

[RFC1995] M. Ohta, "Incremental Zone Transfer in DNS," Internet RFC 1995, Aug. 1996.

[RFC1996] P. Vixie, "A Mechanism for Prompt Notification of Zone Changes (DNS NOTIFY)," Internet RFC 1996, Aug. 1996.

[RFC2136] P. Vixie, ed., S. Thomson, Y. Rekhter, and J. Bound, "Dynamic Updates in the Domain Name System (DNS UPDATE)," Internet RFC 2136, Apr. 1997.

[RFC2141] R. Moats, "URN Syntax," Internet RFC 2141, May 1997.

[RFC2181] R. Elz and R. Bush, "Clarifications to the DNS Specification," Internet RFC 2181, July 1997.

[RFC2308] M. Andrews, "Negative Caching of DNS Queries (DNS NCACHE)," Internet RFC 2308, Mar. 1998.

[RFC2317] H. Eidnes, G. de Groot, and P. Vixie, "Classless IN-ADDR.ARPA Delegation," Internet RFC 2317/BCP 0020, Mar. 1998.

[RFC2606] D. Eastlake 3rd and A. Panitz, "Reserved Top Level DNS Names," Internet RFC 2606/BCP 0032, June 1999.

[RFC2671] P. Vixie, "Extension Mechanisms for DNS (EDNS0)," Internet RFC 2671, Aug. 1999.

[RFC2672] M. Crawford, "Non-Terminal DNS Name Redirection," Internet RFC 2672, Aug. 1999.

[RFC2782] A. Gulbrandsen, P. Vixie, and L. Esibov, "A DNS RR for Specifying the Location of Services (DNS SRV)," Internet RFC 2782, Feb. 2000.

[RFC2845] P. Vixie, O. Gudmundsson, D. Eastlake 3rd, and B. Wellington, "Secret Key Transaction Authentication for DNS (TSIG)," Internet RFC 2845, May 2000.

[RFC2930] D. Eastlake 3rd, "Secret Key Establishment for DNS (TKEY RR)," Internet RFC 2930, Sept. 2000.

[RFC3172] G. Huston, ed., "Management Guidelines and Operational Requirements for the Address and Routing Parameter Area Domain (arpa)," Internet RFC 3172/BCP 0052, Sept. 2001.

[RFC3263] J. Rosenberg and H. Schulzrinne, "Session Initiation Protocol (SIP): Locating SIP Servers," Internet RFC 3263, June 2002.

[RFC3401] M. Mealling, "Dynamic Delegation Discovery System (DDDS)—Part One: The Comprehensive DDDS," Internet RFC 3401 (informational), Oct. 2002.

[RFC3402] M. Mealling, "Dynamic Delegation Discovery System (DDDS)—Part Two: The Algorithm," Internet RFC 3402, Oct. 2002.

[RFC3403] M. Mealling, "Dynamic Delegation Discovery System (DDDS)—Part Three: The Domain Name System (DNS) Database," Internet RFC 3403, Oct. 2002.

[RFC3404] M. Mealling, "Dynamic Delegation Discovery System (DDDS)—Part Four: The Uniform Resource Identifiers (URI) Resolution Application," Internet RFC 3404, Oct. 2002.

[RFC3492] A. Costello, "Punycode: A Bootstring Encoding of Unicode for Internationalized Domain Names in Applications (IDNA)," Internet RFC 3492, Mar. 2003.

[RFC3596] S. Thomson, C. Huitema, V. Ksinant, and M. Souissi, "DNS Extensions to Support IP Version 6," Internet RFC 3596, Oct. 2003.

[RFC3597] A. Gustafsson, "Handling of Unknown DNS Resource Record (RR) Types," Internet RFC 3597, Sept. 2003.

[RFC3833] D. Atkins and R. Austein, "Threat Analysis of the Domain Name System (DNS)," Internet RFC 3833 (informational), Aug. 2004.

[RFC3958] L. Daigle and A. Newton, "Domain-Based Application Service Location Using SRV RRs and the Dynamic Delegation Discovery Service (DDDS)," Internet RFC 3958, Jan. 2005.

[RFC3981] A. Newton and M. Sanz, "IRIS: The Internet Registry Information Service (IRIS) Core Protocol," Internet RFC 3981, Jan. 2005.

[RFC3986] T. Berners-Lee, R. Fielding, and L. Masinter, "Uniform Resource Identifier (URI): Generic Syntax," Internet RFC 3986/STD 0066, Jan. 2005.

[RFC4193] R. Hinden and B. Haberman, "Unique Local IPv6 Unicast Addresses," Internet RFC 4193, Oct. 2005.

[RFC4343] D. Eastlake 3rd, "Domain Name System (DNS) Case Insensitivity Clarification," Internet RFC 4343, Jan. 2006.

[RFC4406] J. Lyon and M. Wong, "Sender ID: Authenticating E-Mail," Internet RFC 4406 (experimental), Apr. 2006.

[RFC4592] E. Lewis, "The Role of Wildcards in the Domain Name System," Internet RFC 4592, July 2006.

[RFC4408] M. Wong and W. Schlitt, "Sender Policy Framework (SPF) for Authorizing Use of Domains in E-Mail, Version 1," Internet RFC 4408 (experimental), Apr. 2006.

[RFC4510] K. Zeilenga, ed., "Lightweight Directory Access Protocol (LDAP): Technical Specification Road Map," Internet RFC 4510, June 2006.

[RFC4690] J. Klensin, P. Falstrom, and C. Karp, "Review and Recommendations for Internationalized Domain Names (IDNs)," Internet RFC 4690 (informational), Sept. 2006.

[RFC4698] E. Gunduz, A. Newton, and S. Kerr, "IRIS: An Address Registry (areg) Type for the Internet Registry Information Service," Internet RFC 4698, Oct. 2006.

[RFC4795] B . Aboba, D. Thaler, and L. Esibov, "Link-Local Multicast Name Resolution (LLMNR)," Internet RFC 4795 (informational), Jan. 2007.

[RFC4848] L. Daigle, "Domain-Based Application Service Location Using URIs and the Dynamic Delegation Discovery Service (DDDS)," Internet RFC 4848, Apr. 2007.

[RFC4960] R. Stewart, ed., "Stream Control Transmission Protocol," Internet RFC 4960, Sept. 2007.

[RFC5001] R. Austein, "DNS Name Server Identifier (NSID) Option," Internet RFC 5001, Aug. 2007.

[RFC5222] T. Hardie et al., "LoST: A Location-to-Service Translation Protocol," Internet RFC 5222, Aug. 2008.

[RFC5321] J. Klensin, "Simple Mail Transfer Protocol," Internet RFC 5321, Oct. 2008.

[RFC5452] A. Hubert and R. van Mook, "Measures for Making DNS More Resilient against Forged Answers," Internet RFC 5452, Jan. 2009.

[RFC5483] L. Conroy and K. Fujiwara, "ENUM Implementation Issues and Experiences," Internet RFC 5483 (informational), Mar. 2009.

[RFC5507] P. Falstrom, R. Austein, and P. Koch, eds., "Design Choices When Expanding the DNS," Internet RFC 5507 (informational), Apr. 2009.

[RFC5509] S. Loreto, "Internet Assigned Numbers Authority (IANA) Registration of Instant Messaging and Presence DNS SRV RRs for the Session Initiation Protocol (SIP)," Internet RFC 5509, Apr. 2009.

[RFC5625] R. Bellis, "DNS Proxy Implementation Guidelines," Internet RFC 5625/BCP 0152, Aug. 2009.

[RFC5782] J. Levine, "DNS Blacklists and Whitelists," Internet RFC 5782 (informational), Feb. 2010.

[RFC5855] J. Abley and T. Manderson, "Nameservers for IPv4 and IPv6 Reverse Zones," Internet RFC 5855/BCP 0155, May 2010.

[RFC5890] J. Klensin, "Internationalized Domain Names for Applications (IDNA): Definitions and Document Framework," Internet RFC 5890, Aug. 2010.

[RFC5891] J. Klensin, "Internationalized Domain Names in Applications (IDNA): Protocol," Internet RFC 5891, Aug. 2010.

[RFC5936] E. Lewis and A. Hoenes, ed., "DNS Zone Transfer Protocol (AXFR)," Internet RFC 5936, June 2010.

[RFC5966] R. Bellis, "DNS Transport over TCP—Implementation Requirements," Internet RFC 5966, Aug. 2010.

[RFC6116] S. Bradner, L. Conroy, and K. Fujiwara, "The E.164 to Uniform Resource Identifiers (URI) Dynamic Delegation Discovery System (DDDS) Application (ENUM)," Internet RFC 6116, Mar. 2011.

[RFC6117] B. Hoeneisen, A. Mayrhofer, and J. Livingood, "IANA Registration of Enumservices: Guide, Template, and IANA Considerations," Internet RFC 6117, Mar. 2011.

[RFC6147] M. Bagnulo, A. Sullivan, P. Matthews, and I. van Beijnum, "DNS64: DNS Extensions for Network Address Translation from IPv6 Clients to IPv4 Servers," Internet RFC 6147, Apr. 2011.

[RFC6168] W. Hardaker, "Requirements for Management of Name Servers for the DNS," Internet RFC 6168 (informational), May 2011.

[RFC6186] C. Daboo, "Use of SRV Records for Locating Email Submission/Access Services," Internet RFC 6186, Mar. 2011.

[RFC6195] D. Eastlake 3rd, "Domain Name System (DNS) IANA Considerations," Internet RFC 6195/BCP 0042, Mar. 2011.

[RFC6303] M. Andrews, "Locally Served DNS Zones," Internet RFC 6303/BCP 0163, July 2011.

[ROOTS] http://www.root-servers.org

[RSYNC] http://rsync.samba.org

[SNP] http://www.iana.org/assignments/s-naptr-parameters

[U11] The Unicode Consortium, *The Unicode Standard, Version 6.0.0* (The Unicode Consortium, 2011).

[URI] http://www.iana.org/assignments/uri-schemes

[URN] http://www.iana.org/assignments/urn-namespaces

[X500] International Telecommunication Union—Telecommunication Standardization Sector, "The Directory—Overview of Concepts, Models and Services," ITU-T X.500, 1993.